

Red Hat Enterprise Linux Administration: Lab Guide

32 hands-on labs, one per chapter. Each one has you build, break and fix a real Red Hat system.

Target: RHEL 10.x Lab: 3 KVM virtual machines Domain: lab.example.test

Network: 192.168.122.0/24

How to use this guide

Each lab pairs with one chapter. Read the chapter first, then do the lab. The tasks show you the key commands, but they are not a script to type blindly. When a task names a command you haven't used, open its manual page before you run it. Most labs end with a **Break it** scenario: you cause a failure on purpose, then find it using the diagnostic methods from the chapter.

A lab is complete only when every item on its **Verify** checklist passes. Check each item with a command. Don't tick it off from memory. If a change has to survive a reboot, reboot and check it again.

The lab environment

You build this in Lab 2. Every later lab refers to these machines by name.

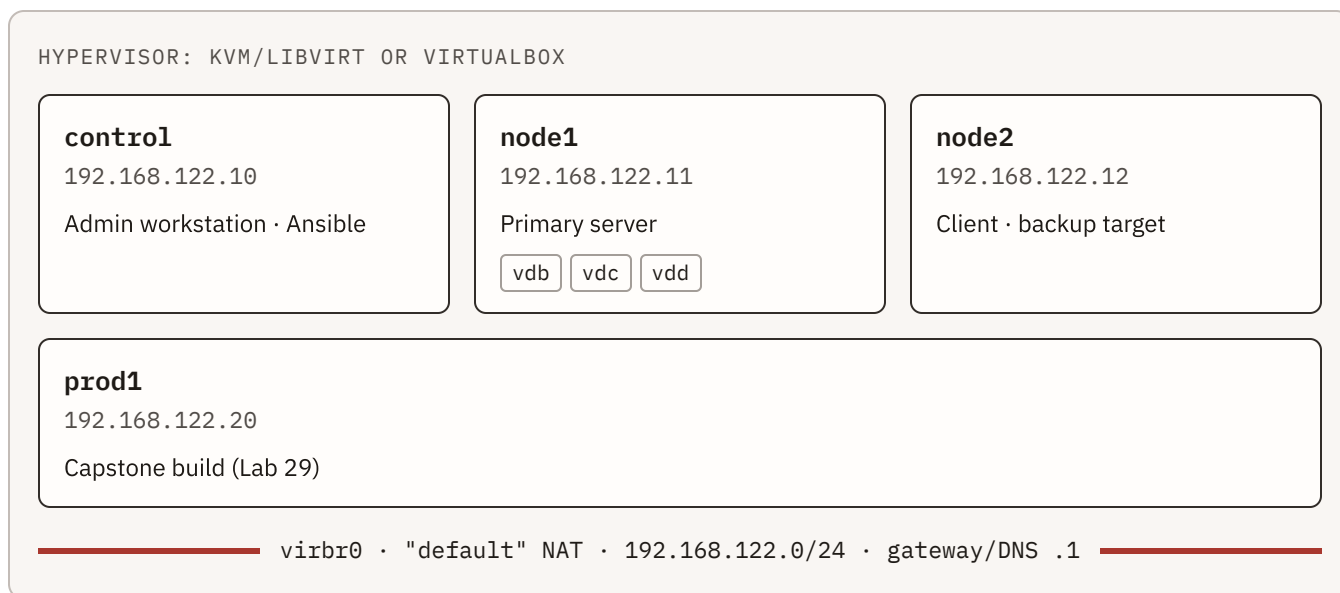


Figure 1. The lab topology. All guests share one NAT network and reach the internet through the hypervisor.

Host	Address	Resources	Role
control	192.168.122.10	2 vCPU · 2 GiB · 20 GiB	Admin workstation, SSH client, Ansible control node
node1	192.168.122.11	2 vCPU · 2 GiB · 20 GiB + 3 × 5 GiB (Lab 12)	Primary server; most labs run here
node2	192.168.122.12	2 vCPU · 2 GiB · 20 GiB	Second server: remote client, backup target, managed node
prod1	192.168.122.20	2 vCPU · 2 GiB · 20 GiB + 2 × 5 GiB	Built from scratch in Lab 29

Addresses assume libvirt's default NAT network. If yours is different, change the addresses to match and keep the host names. Labs 26 and 27 also need an OpenShift environment and a cloud account.

Naming conventions

Use these names exactly. Later labs, scripts and playbooks depend on them.

Thing	Pattern	Examples
Hosts	<code><role><n>.lab.example.test</code>	<code>node1.lab.example.test</code> , <code>prod1</code>
Admin user	<code>student</code> (member of <code>wheel</code>)	<code>student</code>
Test users and groups	short first names; team groups	<code>alice</code> , <code>bob</code> , <code>carol</code> · <code>devs</code> , <code>sysops</code>
VM snapshots	<code>base</code> , then <code>lab-NN</code> before each lab	<code>base</code> , <code>lab-09</code> , <code>lab-24</code>
Drop-in config	<code>NN-lab-<purpose>.conf</code> in the service's <code>.d</code> directory	<code>40-lab-hardening.conf</code> , <code>90-lab.conf</code>
LVM	<code>vg_<purpose></code> / <code>lv_<purpose></code>	<code>vg_lab/lv_app</code> , <code>vg_data/lv_web</code>
Your units and scripts	<code>lab-*</code> or a descriptive verb-noun	<code>lab-uptime.timer</code> , <code>disk-guard.sh</code>
Ansible	<code>~/ansible</code> on control; one playbook per role	<code>inventory.ini</code> , <code>web.yml</code> , <code>site.yml</code>

Conventions

- You work as the `student` user and use `sudo` when a command needs root. The labs never log in as root unless the lab is about root.
- The **Expected output** under a command block is a sample. Versions, IDs, sizes and timestamps on your system will differ. What should match is the shape of the output and the key values.
- In code blocks, lines starting with `#` are comments. Values in `<angle-brackets>` are placeholders you fill in.
- **Snapshot before every lab.** Run `sudo virsh snapshot-create-as node1 lab-NN` on the hypervisor. On VirtualBox, use `VBoxManage snapshot node1 take lab-NN`; to revert, use `VBoxManage snapshot node1 restore lab-NN`. If a lab goes wrong in a way you can't fix, revert. Then try again.
- If you use a free RHEL rebuild (AlmaLinux or Rocky Linux 10) instead of RHEL, the labs work the same. The exceptions are the `subscription-manager` and `Insights` steps.

Contents

PART I: FOUNDATIONS

- Lab 01 What Linux Is and Why Red Hat
- Lab 02 Installing Red Hat and Building Your Lab
- Lab 03 The Command Line and the Shell
- Lab 04 Files, Directories, and the Filesystem Hierarchy
- Lab 05 Text, Editors, and the Unix Philosophy

PART II: THE SYSTEM AND ITS USERS

- Lab 06 Users, Groups, and Permissions
- Lab 07 Privilege, root, and sudo
- Lab 08 Processes and How Programs Run
- Lab 09 systemd and Managing Services
- Lab 10 The Boot Process and System Startup
- Lab 11 Logs, Monitoring, and Reading the System

PART III: STORAGE, NETWORKING, AND SERVICES

- Lab 12 Storage, Disks, and Filesystems
- Lab 13 Logical Volume Management and Flexible Storage
- Lab 14 Networking Fundamentals on Red Hat
- Lab 15 Remote Access and Secure Administration
- Lab 16 Configuring Common Services
- Lab 17 The Firewall and Network Security
- Lab 18 SELinux and Mandatory Access Control

PART IV: ENTERPRISE ADMINISTRATION

- Lab 19 Package Management and Software
- Lab 20 Subscription, Content, and System Management at Scale
- Lab 21 Scheduling, Automation, and Scripting
- Lab 22 Automation at Scale with Ansible
- Lab 23 Backup, Recovery, and System Resilience
- Lab 24 Security Hardening and Compliance

PART V: CONTAINERS AND THE MODERN RED HAT

Lab 25 Containers on Red Hat

Lab 26 OpenShift and Kubernetes on Red Hat

Lab 27 Red Hat in the Cloud and Hybrid Infrastructure

Lab 28 Troubleshooting Red Hat Systems

PART VI: CERTIFICATION, MASTERY, AND CAREER

Lab 29 A Complete Red Hat Build End to End

Lab 30 When Red Hat Systems Fail

Lab 31 Preparing for Red Hat Certification

Lab 32 Building Skills and Where to Go Next

App. A Layered troubleshooting checklist

App. B Command quick reference

What Linux Is and Why Red Hat

Find where Linux already runs in your world, and learn what Red Hat is a version of: the kernel versus the distribution built around it.

SYSTEMS

Any Linux shell (node1 once built)

CHAPTERCh. 1: What Linux Is and Why Red Hat

OBJECTIVES: BY THE END OF THIS LAB

- You can explain the difference between kernel and distribution in one sentence, with a command that shows each
- You can name RHEL's upstream and two free rebuilds
- Your map lists at least three places Linux runs in the cloud or in containers

1 Map Linux around you

Make a list of ten systems you use or depend on: phone, router, TV, streaming service, bank, CI pipeline, cloud VM, containers at work. For each one, write whether it runs the Linux kernel and which category it falls under: device, server, cloud or container. Look up any you aren't sure about.

2 Separate the kernel from the distribution

On any Linux system, check which kernel is running and which distribution is installed. Use the man pages to find out what each field means.

```
uname -srm          # the kernel: name, release, architecture
cat /etc/os-release  # the distribution
rpm -q kernel        # on Red Hat: the packaged kernel(s)
```

EXPECTED OUTPUT

```
Linux 6.12.0-55.9.1.el10_0.x86_64 x86_64
NAME="Red Hat Enterprise Linux"
VERSION="10.0 (Coughlan)"
ID="rhel"
...
kernel-6.12.0-55.9.1.el10_0.x86_64
```

3 Place Red Hat in its family

Draw the upstream-to-downstream flow Fedora → CentOS Stream → RHEL, and show where AlmaLinux and Rocky Linux sit. Next to each one, write one sentence on who it is for. This drawing decides what you install in Lab 2.

4 Find the three ideas

Run each command below. For each one, write one line on which of the three ideas it shows: everything is a file, everything is a process, or everything is text.

```
ls -l /dev | head
ps -e | wc -l
file /etc/passwd /etc/hostname
head -5 /proc/cpuinfo
```

EXPECTED OUTPUT

```
crw-rw-rw-. 1 root root  1,  3 Sep 23 10:02 null
...
148
/etc/passwd:  ASCII text
/etc/hostname: ASCII text
processor    : 0
```

REFLECTION

What does a Red Hat engineer change when they make a RHEL release, and what do they leave to the upstream kernel?

Pick one system from your map. What would you need to know to administer it, and which chapter covers that?

Cleanup. Nothing to clean up.

Installing Red Hat and Building Your Lab

Install RHEL 10 in a virtual machine, choosing each installer option on purpose. Complete the initial setup, take a snapshot, and prove you can break the system and restore it.

SYSTEMS

Hypervisor, control, node1, node2

CHAPTER

Ch. 2: Installing Red Hat and Building Your Lab

OBJECTIVES: BY THE END OF THIS LAB

- All three VMs boot and show the correct hostname in `hostnamectl`
- Each VM has a base snapshot
- After the break, node1 was restored and student can log in again
- subscription-manager status reports the system as registered (RHEL only)

1 Get the media

Sign up for the no-cost Red Hat Developer Subscription at developers.redhat.com. Download the RHEL 10 DVD ISO. RHEL 10 requires an x86-64-v3 CPU, so pass the host CPU through to the VM. If you use a free rebuild instead, download its DVD ISO and skip the registration steps.

2 Create node1

On a Linux hypervisor with KVM/libvirt, run the command below (use `virt-manager` if you prefer a GUI). On VirtualBox, set the CPU to host mode and use `VBoxManage snapshot` for snapshots (see Conventions).

```
sudo dnf -y install @virtualization      # on a RHEL/Fedora host
sudo virt-install --name node1 --memory 2048 --vcpus 2 \
  --cpu host-passthrough --disk size=20 \
  --cdrom ~/iso/rhel-10.0-x86_64-dvd.iso \
  --os-variant rhel10.0 --network network=default
```

EXPECTED OUTPUT

```
Starting install...
Creating domain...
Running graphical console command: virt-viewer --connect qemu:///system --wait node1
```

3 Make each installer choice deliberately

Choose Software Selection: Minimal Install. Choose Installation Destination: Custom. Create /boot (1 GiB, xfs), then an LVM volume group holding / (12 GiB, xfs) and swap (2 GiB). Leave the rest free. Set hostname node1.lab.example.test. Set a root password (Lab 10 needs it on the emergency console). Create the user student and tick "Make this user administrator". Write down why you chose each option.

4 First boot

Log in as student. Register the system, apply updates and confirm the system is what you expect.

```
sudo subscription-manager register --username <rh-developer-login>
sudo dnf -y update
hostnamectl
ip -br addr
id
                                # student should be in wheel
```

EXPECTED OUTPUT

```
The system has been registered with ID: 3f1c7a0e-...
Complete!
  Static hostname: node1.lab.example.test
  Operating System: Red Hat Enterprise Linux 10.0 (Coughlan)
enp1s0    UP    192.168.122.143/24 fe80::5054:ff:fe12:3456/64
uid=1000(student) gid=1000(student) groups=1000(student),10(wheel)
```

5 Snapshot the clean state

Shut the VM down and take a named snapshot. Every later lab starts from a snapshot like this one.

```
sudo virsh shutdown node1
sudo virsh snapshot-create-as node1 base "Clean install, updated"
sudo virsh snapshot-list node1
```

EXPECTED OUTPUT

```
Domain snapshot base created
  Name      Creation Time             State
-----
  base      2026-09-23 10:14:02 +0000  shutoff
```

6 Build node2 and control

Build the other two machines by repeating the install or by cloning node1. A clone needs its own identity: hostname, machine-id and registration.

```
sudo virt-clone --original node1 --name node2 --auto-clone
# then, inside node2:
sudo hostnamectl set-hostname node2.lab.example.test
sudo rm /etc/machine-id && sudo systemd-machine-id-setup
sudo subscription-manager clean && sudo subscription-manager register
```

EXPECTED OUTPUT

```
Allocating 'node1-clone.qcow2'
Clone 'node2' created successfully.
```

BREAK IT · A SYSTEM YOU CAN'T LOG INTO

Break the account database on node1 on purpose, then log out. Try to log in. Then restore the snapshot.

```
sudo mv /etc/passwd /etc/passwd.gone
exit
# on the hypervisor:
sudo virsh destroy node1
sudo virsh snapshot-revert node1 base
sudo virsh start node1
```

REFLECTION

Which installer default would you have accepted without thinking, and what would it have cost you later?

How does the snapshot change the way you are willing to experiment?

Cleanup. Keep all three VMs and their base snapshots. Every later lab depends on them.

The Command Line and the Shell

Work through a set of command-line tasks. Use the manual pages to teach yourself each command instead of being told what to type.

SYSTEMS

node1

CHAPTER

Ch. 3: The Command Line and the Shell

OBJECTIVES: BY THE END OF THIS LAB

- Every answer in task 3 cites the man page or --help section it came from
- You can find a command you have never used with man -k
- HISTTIMEFORMAT persists in a new shell

1 Read the prompt

Find out which shell you are running, who you are, where you are, and how the prompt is built.

```
echo $SHELL; echo $PS1
whoami; hostname; pwd
```

EXPECTED OUTPUT

```
/bin/bash
[\u@\h \w]\$
student
node1.lab.example.test
/home/student
```

2 Take a command apart

For `ls -l -h /var/log`, write down the command name, the options and the argument. Predict whether `ls -lh /var/log` does the same thing. Confirm it.

3 Teach yourself from the manual

Use only `man`, `--help` and `man -k` to answer these: (a) How do you list `/etc` with the newest files first? (b) How do you show the total size of `/var/log` in human-readable units? (c) What does `cp -i` protect you from? (d) Which man section documents the `/etc/passwd` file format? Note where you found each answer.

```
man -k "directory contents"
man ls          # press / to search, n for next match
cp --help | less
man 5 passwd
```

EXPECTED OUTPUT

```
dir (1)  - list directory contents
ls (1)   - list directory contents
vdir (1) - list directory contents
...
```

4 Speed up with history and completion

Rerun commands with history, search back with Ctrl-R, and press Tab twice to see possible completions. Turn on timestamps in history.

```
history | tail -20
!!          # rerun the last command
echo 'export HISTTIMEFORMAT="%F %T "' >> ~/.bashrc && source ~/.bashrc
```

EXPECTED OUTPUT

```
498 2026-09-23 10:31:07 man ls
499 2026-09-23 10:31:40 history | tail -20
```

5 The everyday set

In ~/lab3, use only pwd, cd, ls, mkdir, touch, cp, mv, cat, less, echo, which and type to create, copy, rename and inspect a few files. Use type to explain the difference between cd and ls: one is a shell builtin, the other a program on disk.

BREAK IT · THE OPTION THAT DOES THE OPPOSITE

cp -r src dst behaves differently depending on whether dst already exists. Make both cases and look at what you get. Then find the sentence in man cp that explains it.

```
mkdir -p ~/lab3/src && touch ~/lab3/src/a
cd ~/lab3 && cp -r src dst && ls -R dst    # dst did not exist
cp -r src dst && ls -R dst                # dst existed: now dst/src
```

REFLECTION

Which answer from task 3 would you have guessed wrong without the manual?

How would you find out whether an option exists without running the command?

Cleanup. `rm -rf ~/lab3`

Files, Directories, and the Filesystem Hierarchy

Explore the Red Hat filesystem hierarchy, work with files safely, find files by different criteria, and create both kinds of link so you understand how each one works.

SYSTEMS

node1

CHAPTER

Ch. 4: Files, Directories, and the Filesystem Hierarchy

OBJECTIVES: BY THE END OF THIS LAB

- You can explain why the hard link survived and the symlink broke, in terms of inodes
- Your find commands return results for name, size and content searches
- You can say where Red Hat keeps service config, logs, binaries and user homes

1 Tour the hierarchy

For every top-level directory, write one line on what it holds. Check your answers against `man hier`. Then look at devices and kernel information that appear as files.

```
ls /
man hier
file /dev/vda /dev/null
head -3 /proc/meminfo; cat /proc/sys/kernel/hostname
```

EXPECTED OUTPUT

```
afs bin boot dev etc home lib lib64 media mnt opt proc root run sbin srv
sys tmp usr var
/dev/vda: block special (252/0)
/dev/null: character special (1/3)
MemTotal:          1983452 kB
node1.lab.example.test
```

2 Find where a package lives

Ask the package database where OpenSSH keeps its configuration, binaries and documentation.

```
rpm -ql openssh-server | grep -E "^/etc|bin/"
```

EXPECTED OUTPUT

```
/etc/pam.d/sshd
/etc/ssh/sshd_config
/etc/ssh/sshd_config.d
/usr/sbin/sshd
...
```

3 Work carefully

Create a practice tree. Copy, move and delete in it using the interactive flags. Before any wildcard delete, preview it with echo.

```
mkdir -p ~/lab4/{src,dst}
touch ~/lab4/src/file{1..5}.txt
cp -iv ~/lab4/src/file1.txt ~/lab4/dst/
mv -iv ~/lab4/src/file2.txt ~/lab4/dst/renamed.txt
echo rm ~/lab4/src/*.txt      # preview first
rm -Iv ~/lab4/src/*.txt
```

EXPECTED OUTPUT

```
'/home/student/lab4/src/file1.txt' -> '/home/student/lab4/dst/file1.txt'
renamed '/home/student/lab4/src/file2.txt' -> '/home/student/lab4/dst/renamed.txt'
rm /home/student/lab4/src/file3.txt /home/student/lab4/src/file4.txt
/home/student/lab4/src/file5.txt
```

4 Absolute and relative paths

Move around using relative paths only, and predict where you will end up before each cd. Confirm with pwd and readlink -f.

```
cd /usr/share/doc
cd ../../bin && pwd
readlink -f ../lib64
```

EXPECTED OUTPUT

```
/usr/bin
/usr/lib64
```

5 Find files

Find files by name, type, size, age and content.

```
find /etc -name "*.conf" -newer /etc/hostname
sudo find /var -xdev -type f -size +10M
sudo find /etc -type f -exec grep -l "PermitRootLogin" {} +
```

EXPECTED OUTPUT

```
/etc/sysctl.conf
/etc/ssh/sshd_config
```

6 Hard and symbolic links

Create both kinds of link, compare their inode numbers, then delete the original and see what still works.

```
echo "original" > ~/lab4/orig
ln ~/lab4/orig ~/lab4/hard
ln -s ~/lab4/orig ~/lab4/soft
ls -li ~/lab4
rm ~/lab4/orig; cat ~/lab4/hard; cat ~/lab4/soft
```

EXPECTED OUTPUT

```
1310772 -rw-r--r--. 2 student student 9 Sep 23 10:40 hard
1310772 -rw-r--r--. 2 student student 9 Sep 23 10:40 orig
1310773 lrwxrwxrwx. 1 student student 25 Sep 23 10:40 soft -> /home/student/lab4/orig
original
cat: /home/student/lab4/soft: No such file or directory
```

BREAK IT • THE STRAY SPACE

Preview what a one-character typo would do. Don't run it without echo.

```
cd ~/lab4
echo rm -rf ~/lab4/dst /*      # note the space before /*
```

REFLECTION

Why does the filesystem hierarchy put configuration, binaries and variable data in separate trees?

When would you choose a hard link over a symbolic link?

Cleanup. `rm -rf ~/lab4`

Text, Editors, and the Unix Philosophy

Edit a real configuration file safely. Then build a pipeline of small tools that answers real questions about the system.

SYSTEMS

node1

CHAPTER

Ch. 5: Text, Editors, and the Unix Philosophy

OBJECTIVES: BY THE END OF THIS LAB

- dnf works and dnf.conf contains your intended change
- Your shell-count pipeline agrees with a manual count of /etc/passwd
- You can explain the difference between 2> and 2>&1

1 Learn one editor properly

Work through vimtutor (about 30 minutes) or pick nano. After that you should be able to open, search, change, save and quit without help.

```
vimtutor
```

2 Read without editing

Use each viewing tool on a real file and note when you would choose it.

```
less /etc/services
head -n 5 /etc/passwd; tail -n 5 /etc/passwd
wc -l /etc/services
sudo tail -f /var/log/messages      # Ctrl-C to stop
```

EXPECTED OUTPUT

```
15073 /etc/services
```

3 Edit a configuration file safely

Back up the file, make the change, check that the tool accepts it, and diff against the backup.

```
sudo cp -a /etc/dnf/dnf.conf /etc/dnf/dnf.conf.bak
sudo vim /etc/dnf/dnf.conf      # add: max_parallel_downloads=10
dnf config-manager --dump | grep parallel
diff /etc/dnf/dnf.conf.bak /etc/dnf/dnf.conf
```

EXPECTED OUTPUT

```
max_parallel_downloads = 10
4a5
> max_parallel_downloads=10
```

4 Pipes: answer a question

Which login shells are in use, and by how many accounts? Build the pipeline one stage at a time and look at the output after each stage.

```
cut -d: -f7 /etc/passwd | sort | uniq -c | sort -rn
```

EXPECTED OUTPUT

```
3 /sbin/nologin
1 /bin/bash
...
```

5 Redirection

Send normal output and errors to different files, then combine them and use tee.

```
find /etc -name "*.repo" > repos.txt 2> errors.txt
wc -l repos.txt errors.txt
find /etc -name "*.repo" 2>&1 | tee all.txt | wc -l
```

EXPECTED OUTPUT

```
4 repos.txt
0 errors.txt
4
```

6 Search and transform

Find the five largest installed packages, formatted in MB. Then count failed and accepted SSH logins.

```
rpm -qa --qf "%{SIZE} %{NAME}\n" | sort -rn | head -5 \
| awk '{printf "%6.1f MB  %s\n", $1/1048576, $2}'
sudo grep -cE "Failed password" /var/log/secure
sudo grep -wE "Accepted" /var/log/secure | tail -3
```

EXPECTED OUTPUT

```
512.3 MB  linux-firmware
148.9 MB  glibc-all-langpacks
...
0
Sep 23 10:02:11 node1 sshd[1532]: Accepted password for student from 192.168.122.1 port
51422 ssh2
```

BREAK IT • THE BAD EDIT

Break the section header in dnf.conf, for example change [main] to [main. Run any dnf command and read the error. Then find the damage with diff and restore the backup.

```
sudo sed -i "s/^\[main\]/[main/" /etc/dnf/dnf.conf
dnf repolist
diff /etc/dnf/dnf.conf.bak /etc/dnf/dnf.conf
sudo cp -a /etc/dnf/dnf.conf.bak /etc/dnf/dnf.conf
```

REFLECTION

Which stage of your pipeline did the most work, and could you replace it with a different tool?

What is the minimum routine you will follow before editing any configuration file?

Cleanup. `rm -f ~/repos.txt ~/errors.txt ~/all.txt`

Users, Groups, and Permissions

Create users and groups, set up a shared directory with the right permissions, and fix a permission-denied problem by working through the permission model.

SYSTEMS

node1

CHAPTER

Ch. 6: Users, Groups, and Permissions

OBJECTIVES: BY THE END OF THIS LAB

- alice and bob can each edit files in /srv/devs; carol gets permission denied
- bob can't delete alice's file (sticky bit)
- New files in /srv/devs belong to group devs

1 Create identities

Create a team group with two members, plus one outsider.

```
sudo groupadd devs
sudo useradd -m -G devs alice
sudo useradd -m -G devs bob
sudo useradd -m carol
for u in alice bob carol; do sudo passwd $u; done
id alice; getent group devs
```

EXPECTED OUTPUT

```
uid=1001(alice) gid=1001(alice) groups=1001(alice),1000(devs)
devs:x:1000:alice,bob
```

2 Read the model

Read the permissions on these files and explain each set of bits, both symbolic and octal.

```
stat -c "%A %a %U:%G %n" /etc/shadow /usr/bin/passwd /tmp /home/alice
```

EXPECTED OUTPUT

```
----- 0 root:root /etc/shadow
-rwsr-xr-x 4755 root:root /usr/bin/passwd
drwxrwxrwt 1777 root:root /tmp
drwx----- 700 alice:alice /home/alice
```

3 Build a shared directory

devs members can read and write each other's files in /srv/devs. New files inherit the devs group. Nobody can delete another member's files. carol has no access at all.

```
sudo mkdir /srv/devs
sudo chown root:devs /srv/devs
sudo chmod 3770 /srv/devs      # setgid + sticky + rwxrwx---
sudo -u alice touch /srv/devs/a.txt && ls -l /srv/devs
```

EXPECTED OUTPUT

```
-rw-r--r--. 1 alice devs 0 Sep 23 11:02 a.txt
```

4 Audit special permissions

List the setuid and setgid programs on the system. Pick one and explain why it needs the bit.

```
sudo find / -xdev -type f \( -perm -4000 -o -perm -2000 \) -exec ls -l {} +
```

EXPECTED OUTPUT

```
-rwsr-xr-x. 1 root root  33424 ... /usr/bin/passwd
-rwsr-xr-x. 1 root root  ... /usr/bin/sudo
...
```

5 Diagnose permission denied

alice creates a file bob can't read. Walk the whole path with namei, find which bit blocks bob, fix it, and explain the fix.

```
sudo -u alice sh -c 'umask 077; echo plan > /srv/devs/plan.txt'
sudo -u bob cat /srv/devs/plan.txt
namei -l /srv/devs/plan.txt
sudo -u alice chmod 660 /srv/devs/plan.txt
```

EXPECTED OUTPUT

```
cat: /srv/devs/plan.txt: Permission denied
f: /srv/devs/plan.txt
dr-xr-xr-x root  root  /
drwxr-xr-x root  root  srv
drwxrws--T root  devs  devs
-rw----- alice devs  plan.txt
```

BREAK IT · THE WORLD-WRITABLE HOLE

Make a file world-writable, then find every world-writable file on the root filesystem the way an auditor would.

```
sudo touch /usr/local/bin/helper && sudo chmod 777 /usr/local/bin/helper  
sudo find / -xdev -type f -perm -0002 -ls  
sudo rm /usr/local/bin/helper
```

REFLECTION

In the permission-denied case, which part of the path did you check first, and was that the right place to start?

What does the setgid bit on a directory give a team that careful umask settings alone don't?

Cleanup. Keep alice, bob and /srv/devs. Later labs use them.

Privilege, root, and sudo

Configure sudo so a user can run specific administrative commands. Confirm the boundary holds, and audit what privilege has been handed out.

SYSTEMS

node1

CHAPTER

Ch. 7: Privilege, root, and sudo

OBJECTIVES: BY THE END OF THIS LAB

- `sudo -l -U alice` lists only the four WEBSVC commands
- The refused attempts appear in `/var/log/secure`
- `visudo -c` reports all files parsed OK

1 Know who holds power

List every account that can become root and every sudo rule currently in effect.

```
getent group wheel
sudo -l
sudo grep -rhvE "^s*(#|$)" /etc/sudoers /etc/sudoers.d/
```

EXPECTED OUTPUT

```
wheel:x:10:student
User student may run the following commands on node1:
(ALL) ALL
```

2 Delegate precisely

Let alice manage the web service and do nothing else. Always edit sudoers with visudo.

```
sudo visudo -f /etc/sudoers.d/40-webops
# contents:
Cmd_Alias WEBSVC = /usr/bin/systemctl start httpd, \
    /usr/bin/systemctl stop httpd, /usr/bin/systemctl restart httpd, \
    /usr/bin/systemctl status httpd
alice ALL=(root) WEBSVC

sudo visudo -c
```

EXPECTED OUTPUT

```
/etc/sudoers: parsed OK
/etc/sudoers.d/40-webops: parsed OK
```

3 Test the boundary

As alice, confirm the allowed commands work and that everything else is refused, including becoming root.

```
sudo -l -U alice
su - alice
sudo systemctl status httpd      # allowed
sudo systemctl restart sshd      # refused
sudo -i                          # refused
```

EXPECTED OUTPUT

```
User alice may run the following commands on node1:
  (root) /usr/bin/systemctl start httpd, /usr/bin/systemctl stop httpd, ...
Sorry, user alice is not allowed to execute '/bin/systemctl restart sshd' as root on
node1.
```

4 Recognise an escalation path

Explain why the rule `alice ALL=(root) /usr/bin/vim` is the same as giving alice full root. From vim, `!sh` gives a root shell. Don't add this rule. Write down the safer alternative instead: `sudoedit` with a specific file path.

5 Audit use of privilege

Find every sudo use, and every refusal, since the lab started.

```
sudo journalctl _COMM=sudo --since today --no-pager | tail
sudo grep -E "command not allowed|NOT in sudoers" /var/log/secure
```

EXPECTED OUTPUT

```
Sep 23 11:20:14 node1 sudo[2210]:  alice : command not allowed ; TTY=pts/1 ;
PWD=/home/alice ; USER=root ; COMMAND=/bin/systemctl restart sshd
```

BREAK IT • THE UNVALIDATED SUDOERS FILE

Snapshot first. Write a broken rule without `visudo` and watch sudo stop working for everyone. Recover with `su -` using the root password from Lab 2, or revert the snapshot.

```
echo "alice ALL=(root) | sudo tee /etc/sudoers.d/99-bad
sudo -l                                # sudo is now broken
su - -c "rm /etc/sudoers.d/99-bad"
```

REFLECTION

What is the practical difference between "alice can run systemctl" and "alice can run systemctl restart httpd"?

How would you find out, six months from now, who granted a sudo rule and why?

Cleanup. Keep /etc/sudoers.d/40-webops. Remove /etc/sudoers.d/99-bad if it is still there.

Processes and How Programs Run

View and control processes, and run and manage background jobs. Find a process that is using too many resources and stop it safely.

SYSTEMS

node1 (two terminals)

CHAPTER

Ch. 8: Processes and How Programs Run

OBJECTIVES: BY THE END OF THIS LAB

- You found both runaway processes without looking at terminal 2
- Load average and free memory went back to normal after you stopped them
- jobs shows no leftover jobs

1 See what is running

Look at the process tree, your own shell, and a live view. In top, try P, M, k, r and q.

```
ps -ef --forest | less
pstree -p | head -30
ps -o pid,ppid,stat,ni,%cpu,%mem,cmd -p $$
top
```

EXPECTED OUTPUT

PID	PPID	STAT	NI	%CPU	%MEM	CMD
1874	1873	Ss	0	0.0	0.2	-bash

2 Manage jobs in the shell

Run jobs in the background, suspend them, then bring them back to the foreground.

```
sleep 600 &
sleep 700          # then press Ctrl-Z
jobs -l
bg %2; fg %1       # then Ctrl-C
nohup sleep 900 > /dev/null 2>&1 & disown
```

EXPECTED OUTPUT

[1]-	2301	Running	sleep 600 &
[2]+	2302	Stopped	sleep 700

3 Send signals

Pause, resume and stop a process. Watch the STAT column change as you do.

```
sleep 900 & P=$!  
kill -STOP $P; ps -o pid,stat,cmd -p $P  
kill -CONT $P; ps -o pid,stat,cmd -p $P  
kill -TERM $P  
kill -l
```

EXPECTED OUTPUT

```
PID STAT CMD  
2310 T    sleep 900  
2310 S    sleep 900
```

4 Influence priority

Start two CPU-bound processes, one at low priority. Compare their CPU share in top, then renice one.

```
nice -n 15 sha1sum /dev/zero &  
sha1sum /dev/zero &  
top -o %CPU          # compare NI and %CPU  
sudo renice -n 5 -p <pid>
```

EXPECTED OUTPUT

```
PID USER  PR  NI  %CPU COMMAND  
2320 student 20   0 97.0 sha1sum  
2319 student 35  15  3.0 sha1sum
```

BREAK IT • THE RUNAWAY PROCESS

In the second terminal, start a CPU hog and a process holding 1.5 GiB of memory. In the first terminal, find both using only diagnostic tools. Stop each with SIGTERM first. Use SIGKILL only if the process ignores TERM.

```
# terminal 2:  
cat /dev/urandom | gzip -9 > /dev/null &  
python3 -c "a = bytearray(1500 * 1024 * 1024); input()"  
# terminal 1:  
ps -eo pid,user,%cpu,%mem,cmd --sort=-%cpu | head -5  
ps -eo pid,user,%mem,rss,cmd --sort=-%mem | head -5  
free -h; uptime
```

REFLECTION

*Why send **TERM** before **KILL**? What does the process lose when it gets **KILL**?*

Which command found the runaway process fastest, and why?

Cleanup. pkill sha1sum; pkill -f "sleep 9"

systemd and Managing Services

Manage services with systemd, make one survive a reboot, schedule a task with a timer, and find out why a service won't start.

SYSTEMS

node1

CHAPTER

Ch. 9: systemd and Managing Services

OBJECTIVES: BY THE END OF THIS LAB

- httpd is active and enabled after a reboot
- /var/log/lab-uptime.log gains a line every five minutes
- labd is active, and you can explain what 203/EXEC means

1 Explore units

List loaded services, enabled unit files, and the full unit definition of sshd.

```
systemctl list-units --type=service
systemctl list-unit-files --state=enabled
systemctl cat sshd
```

2 Run and persist a service

Install httpd and start it. Reboot, and notice that started does not mean enabled. Then enable it and reboot again.

```
sudo dnf -y install httpd
sudo systemctl start httpd; systemctl is-enabled httpd
sudo systemctl reboot
systemctl is-active httpd          # inactive
sudo systemctl enable --now httpd
```

EXPECTED OUTPUT

```
disabled
inactive
(after enable --now): Created symlink /etc/systemd/system/multi-
user.target.wants/httpd.service → /usr/lib/systemd/system/httpd.service.
```

3 Read dependencies

Find what httpd needs, what needs it, and the order they start in.

```
systemctl list-dependencies httpd | head
systemctl show -p Requires,Wants,After httpd
```

EXPECTED OUTPUT

```
httpd.service
• └─.mount
• └─system.slice
• └─sysinit.target
Requires=-.mount system.slice
After=network.target remote-fs.target nss-lookup.target ...
```

4 Schedule with a timer

Record the load average every five minutes using a oneshot service and a timer.

```
sudo tee /etc/systemd/system/lab-uptime.service <<'EOF'
[Unit]
Description=Record load average
[Service]
Type=oneshot
ExecStart=/usr/bin/sh -c "uptime >> /var/log/lab-uptime.log"
EOF
sudo tee /etc/systemd/system/lab-uptime.timer <<'EOF'
[Unit]
Description=Record load every 5 minutes
[Timer]
OnCalendar=*:0/5
Persistent=true
[Install]
WantedBy=timers.target
EOF
sudo systemctl daemon-reload
sudo systemctl enable --now lab-uptime.timer
systemctl list-timers lab-uptime.timer
```

EXPECTED OUTPUT

NEXT	LEFT	LAST	PASSED	UNIT	ACTIVATES
Wed 2026-09-23 11:45:00 UTC	3min 2s	-	-	lab-uptime.timer	lab-uptime.service

5 Diagnose a service that won't start

Create a unit that points to a missing binary. Read status and the journal, find the cause, and fix it.

```
sudo tee /etc/systemd/system/labd.service <<'EOF'
[Service]
ExecStart=/usr/local/bin/labd
[Install]
WantedBy=multi-user.target
EOF
sudo systemctl daemon-reload; sudo systemctl start labd
systemctl status labd          # note status=203/EXEC
journalctl -u labd -n 20 --no-pager
printf "#!/bin/bash\nexec sleep infinity\n" | sudo tee /usr/local/bin/labd
sudo chmod 755 /usr/local/bin/labd && sudo systemctl start labd
```

EXPECTED OUTPUT

```
× labd.service
   Active: failed (Result: exit-code)
   Process: 3012 ExecStart=/usr/local/bin/labd (code=exited, status=203/EXEC)
 labd.service: Failed to locate executable /usr/local/bin/labd: No such file or directory
```

BREAK IT • THE MASKED SERVICE

Have a partner, or your past self, run `sudo systemctl mask httpd` without telling you. Diagnose why `httpd` won't start and why `enable` fails.

```
sudo systemctl mask httpd
sudo systemctl start httpd
systemctl status httpd; ls -l /etc/systemd/system/httpd.service
sudo systemctl unmask httpd
```

REFLECTION

What is the difference between active and enabled? Which one would an exam check after a reboot?

What does a systemd timer give you that cron doesn't?

Cleanup. `sudo systemctl disable --now labd lab-uptime.timer; sudo rm /etc/systemd/system/labd.service`

The Boot Process and System Startup

Trace the boot process, break the boot on purpose and recover using rescue tools. Then work out which boot stage a staged failure happened at.

SYSTEMS

node1 (console access)

CHAPTER

Ch. 10: The Boot Process and System Startup

OBJECTIVES: BY THE END OF THIS LAB

- node1 boots cleanly to multi-user.target with no failed units
- You can match three symptoms to their stages: grub> prompt, kernel panic on root mount, emergency shell
- You changed the root password and SELinux relabelled on boot

1 Trace a boot

Take a snapshot first. Measure the boot, find the slowest units, and read the earliest journal entries.

```
systemd-analyze
systemd-analyze blame | head
systemd-analyze critical-chain
journalctl -b -o short-monotonic | head -40
journalctl --list-boots
```

EXPECTED OUTPUT

```
Startup finished in 1.9s (kernel) + 2.3s (initrd) + 6.1s (userspace) = 10.3s
multi-user.target reached after 6.0s in userspace.
```

2 Inspect the bootloader

See which kernel GRUB boots by default and which arguments it passes.

```
sudo grubby --default-kernel
sudo grubby --info=ALL | grep -E "^(kernel|args)"
ls /boot/loader/entries/
```

EXPECTED OUTPUT

```
/boot/vmlinuz-6.12.0-55.9.1.el10_0.x86_64
kernel="/boot/vmlinuz-6.12.0-55.9.1.el10_0.x86_64"
args="ro crashkernel=... resume=/dev/mapper/rhel-swap rd.lvm.lv=rhel/root ..."
```

3 Control the target

Show the default target and list the targets that are active now.

```
systemctl get-default
systemctl list-units --type=target
sudo systemctl isolate multi-user.target
```

EXPECTED OUTPUT

```
multi-user.target
```

4 Break and recover: fstab

Add an fstab entry for a device that doesn't exist, then reboot. After a timeout of about 90 seconds you land in emergency mode. Log in with the root password and repair the entry.

```
echo "UUID=00000000-dead-beef /mnt/ghost xfs defaults 0 0" | sudo tee -a /etc/fstab
sudo systemctl reboot
# at the emergency prompt:
mount -o remount,rw /
vi /etc/fstab          # delete the ghost line
systemctl daemon-reload && reboot
```

EXPECTED OUTPUT

```
You are in emergency mode. After logging in, type "journalctl -xb" to view system logs
...
Give root password for maintenance
(or press Control-D to continue):
```

5 Reset a lost root password

At the GRUB menu, press e. Add rd.break to the end of the linux line and press Ctrl-X. This drops you into the initramfs before the real root is mounted read-write.

```
mount -o remount,rw /sysroot
chroot /sysroot
passwd root
touch /.autorelabel      # relabel SELinux contexts on next boot
exit; exit
```

BREAK IT · WHICH STAGE FAILED?

Move the running kernel's initramfs away and reboot. Identify the stage from the symptom: bootloader, kernel/initramfs, or systemd. Recover by choosing the rescue kernel entry in GRUB and putting the file back.

```
sudo mv /boot/initramfs-$(uname -r).img /root/  
sudo systemctl reboot  
# boot the "rescue" entry, then:  
sudo mv /root/initramfs-*.img /boot/ && sudo restorecon -v /boot/initramfs-*
```

REFLECTION

For each break in this lab, which stage failed? Which log or symptom told you?

Why is touch /.autorelabel needed after a password reset in rd.break?

Cleanup. If anything is still broken: `sudo virsh snapshot-revert node1 lab-10`

Logs, Monitoring, and Reading the System

Read and filter the systemd journal and the log files, watch system resources under load, and use the logs to diagnose a staged problem.

SYSTEMS

node1, node2

CHAPTER

Ch. 11: Logs, Monitoring, and Reading the System

OBJECTIVES: BY THE END OF THIS LAB

- You can report the attempt count, usernames tried and source IP from the logs alone
- The LAB11 marker appears in both the journal and /var/log/messages
- You can say which tool you would use for CPU, memory, disk and load

1 Filter the journal

Filter the journal by priority, unit, time, process and field.

```
journalctl -b -p err --no-pager
journalctl -u sshd --since "1 hour ago"
journalctl _PID=1 -n 20
journalctl -o verbose -n 1
journalctl -f          # follow live, Ctrl-C to stop
```

EXPECTED OUTPUT

```
Sep 23 09:58:02 node1 kernel: ...
Sep 23 10:02:11 node1 sshd[1532]: Server listening on 0.0.0.0 port 22.
```

2 Confirm the journal persists

Check that journal entries survive a reboot, and how much disk space the journal uses.

```
ls -ld /var/log/journal
journalctl --list-boots
journalctl --disk-usage
```

EXPECTED OUTPUT

```
drwxr-sr-x+. 3 root systemd-journal 46 Sep 23 09:58 /var/log/journal
IDX BOOT ID                FIRST ENTRY  LAST ENTRY
-1 7c1e...                 Wed ...     Wed ...
 0 a90d...                 Wed ...     Wed ...
Archived and active journals take 24M in the file system.
```

3 Traditional log files

Write a tagged marker message, then find it in both the text logs and the journal.

```
ls -lt /var/log | head
logger -t lab11 -p user.warning "LAB11 marker"
sudo grep LAB11 /var/log/messages
journalctl -t lab11
```

EXPECTED OUTPUT

```
Sep 23 12:04:51 node1 lab11[3301]: LAB11 marker
```

4 Monitor under load

Create CPU load and watch it with several tools. Note which tool answers which question.

```
sudo dnf -y install sysstat
for i in 1 2; do shasum /dev/zero & done
uptime; vmstat 2 5; mpstat -P ALL 2 3
free -h; df -h
pkill shasum
```

EXPECTED OUTPUT

```
procs -----memory----- --swap-- ----io---- -system-- -----cpu-----
 r  b   swpd   free   buff  cache   si   so    bi    bo    in   cs us sy id wa st
 2  0       0 1102340   4212 520112    0    0     1     3  812  401 99  1  0  0  0
```

5 Identify the system

Collect a hardware and state profile of node1.

```
lscpu | head -15; lsblk; lsmem | tail -3
hostnamectl; cat /etc/redhat-release
```

BREAK IT • THE UNEXPLAINED LOGINS

From node2, try several SSH logins to node1 with wrong passwords and one non-existent user. On node1, use only the logs to work out what happened, when, and from which address.

```
# on node2:
for i in 1 2 3; do ssh nosuch@192.168.122.11 true; done
# on node1:
sudo journalctl -u sshd --since "15 min ago" | grep -E "Invalid user|Failed"
sudo grep "Failed password" /var/log/secure | awk '{print $(NF-3)}' | sort | uniq -c
```

REFLECTION

Which log contained the evidence first, the journal or /var/log/secure? Why do both exist?

What would you monitor on a production web server, and why those metrics?

Cleanup. pkill sha1sum. Keep sysstat installed.

Storage, Disks, and Filesystems

Add a disk to your lab system, partition it, create a filesystem and mount it so it persists. Then find what filled up a full filesystem.

SYSTEMS

Hypervisor, node1

CHAPTER

Ch. 12: Storage, Disks, and Filesystems

OBJECTIVES: BY THE END OF THIS LAB

- /data and the new swap survive a reboot
- findmnt --verify reports no errors
- You recovered all three sources of usage; df and du agree afterwards

1 Add three disks

On the hypervisor, attach three 5 GiB disks to node1. vdb is for this lab; vdc and vdd are for Lab 13.

```
for d in vdb vdc vdd; do
    sudo qemu-img create -f qcow2 /var/lib/libvirt/images/node1-$d.qcow2 5G
    sudo virsh attach-disk node1 /var/lib/libvirt/images/node1-$d.qcow2 $d \
        --driver qemu --subdriver qcow2 --persistent
done
# on node1:
lsblk
```

EXPECTED OUTPUT

```
vda      252:0      0  20G  0 disk
├─vda1   252:1      0   1G  0 part /boot
└─vda2   252:2      0  19G  0 part
vdb      252:16     0   5G  0 disk
vdc      252:32     0   5G  0 disk
vdd      252:48     0   5G  0 disk
```

2 Partition

Create a GPT label with a 3 GiB data partition and a 1 GiB swap partition.

```
sudo parted /dev/vdb mklabel gpt
sudo parted /dev/vdb mkpart data xfs 1MiB 3GiB
sudo parted /dev/vdb mkpart swap linux-swaps 3GiB 4GiB
sudo udevadm settle; lsblk /dev/vdb
```

EXPECTED OUTPUT

```
vdb      252:16    0    5G    0 disk
├─vdb1  252:17    0    3G    0 part
└─vdb2  252:18    0    1G    0 part
```

3 Create and mount persistently

Make an XFS filesystem and mount it by UUID. Before you reboot, check fstab.

```
sudo mkfs.xfs -L data /dev/vdb1
sudo mkdir /data
U=$(sudo blkid -s UUID -o value /dev/vdb1)
echo "UUID=$U /data xfs defaults 0 0" | sudo tee -a /etc/fstab
sudo systemctl daemon-reload && sudo mount -a
sudo findmnt --verify; findmnt /data
```

EXPECTED OUTPUT

```
Success, no errors or warnings detected
TARGET SOURCE      FSTYPE OPTIONS
/data  /dev/vdb1 xfs      rw,relatime,seclabel,attr2,inode64,...
```

4 Add swap

Format the swap partition, add it to fstab and turn it on.

```
sudo mkswap /dev/vdb2
U=$(sudo blkid -s UUID -o value /dev/vdb2)
echo "UUID=$U none swap defaults 0 0" | sudo tee -a /etc/fstab
sudo swapon -a; swapon --show; free -h
```

EXPECTED OUTPUT

```
NAME      TYPE      SIZE USED PRIO
/dev/dm-1 partition 2G   0B   -2
/dev/vdb2 partition 1024M 0B   -3
```

5 Prove persistence

Reboot. Then confirm /data is mounted and both swap devices are active.

```
sudo systemctl reboot
findmnt /data; swapon --show
```

EXPECTED OUTPUT

```
TARGET SOURCE      FSTYPE OPTIONS
/data  /dev/vdb1 xfs      rw,relatime,seclabel,...
```

BREAK IT · THE FULL FILESYSTEM

Fill /data in three different ways, one of them hidden and one held by a deleted file that is still open. Find all three using du and lsof.

```
sudo fallocate -l 1500M /data/bigfile
sudo mkdir -p /data/app/.cache && sudo fallocate -l 700M /data/app/.cache/tmp.bin
sudo sh -c "exec 3>/data/ghost.log; fallocate -l 400M /data/ghost.log; rm
/data/ghost.log; sleep 3600" &
df -h /data
sudo du -xh --max-depth=3 /data | sort -h | tail
sudo dnf -y install lsof && sudo lsof +L1 /data
```

REFLECTION

Why does fstab use UUIDs instead of device names like /dev/vdb1?

How did df and du disagree in the Break it scenario, and what caused the gap?

Cleanup. `sudo rm -rf /data/bigfile /data/app`; stop the ghost process with `kill <pid>`

Logical Volume Management and Flexible Storage

Build logical volumes across more than one disk, grow a filesystem that has run out of space, and take and use an LVM snapshot.

SYSTEMS node1	CHAPTER Ch. 13: Logical Volume Management and Flexible Storage
------------------	---

OBJECTIVES: BY THE END OF THIS LAB

- /app is 3 GiB and mounts at boot
- The merge restored version.txt to v1
- vgs shows free extents left for future growth

1 Build the stack

Turn two disks into physical volumes, pool them into a volume group, and check each layer.

```
sudo pvcreate /dev/vdc /dev/vdd
sudo vgcreate vg_lab /dev/vdc /dev/vdd
sudo pvs; sudo vgs
```

EXPECTED OUTPUT

```
Physical volume "/dev/vdc" successfully created.
Volume group "vg_lab" successfully created
VG      #PV #LV #SN Attr   VSize VFree
vg_lab   2   0   0 wz--n- 9.99g 9.99g
```

2 Create and mount a logical volume

Create a 1 GiB volume, format it and mount it at /app so it persists.

```
sudo lvcreate -n lv_app -L 1G vg_lab
sudo mkfs.xfs /dev/vg_lab/lv_app
sudo mkdir /app
echo "/dev/vg_lab/lv_app /app xfs defaults 0 0" | sudo tee -a /etc/fstab
sudo systemctl daemon-reload && sudo mount -a && df -h /app
```

EXPECTED OUTPUT

```
Filesystem                Size  Used Avail Use% Mounted on
/dev/mapper/vg_lab-lv_app 960M   39M  922M   5% /app
```

3 Run out, then grow

Fill the volume, then grow the volume and its filesystem in one step, with no downtime. XFS can grow but can't shrink. Write down what that means for how you size volumes.

```
sudo fallocate -l 950M /app/fill.bin; df -h /app
sudo lvextend -r -L +2G /dev/vg_lab/lv_app
df -h /app; sudo lvs
```

EXPECTED OUTPUT

```
/dev/mapper/vg_lab-lv_app 960M 989M 0 100% /app
Logical volume vg_lab/lv_app successfully resized.
data blocks changed from 262144 to 786432
/dev/mapper/vg_lab-lv_app 3.0G 1010M 2.0G 34% /app
```

4 Snapshot and roll back

Take a snapshot, damage the original, look at the snapshot, then merge it back.

```
echo v1 | sudo tee /app/version.txt
sudo lvcreate -s -n lv_app_snap -L 500M /dev/vg_lab/lv_app
echo v2-broken | sudo tee /app/version.txt
sudo mkdir -p /mnt/snap && sudo mount -o ro,nouuid /dev/vg_lab/lv_app_snap /mnt/snap
cat /mnt/snap/version.txt; sudo lvs      # watch Data%
sudo umount /mnt/snap /app
sudo lvconvert --merge /dev/vg_lab/lv_app_snap
sudo mount /app && cat /app/version.txt
```

EXPECTED OUTPUT

```
v1
LV          VG      Attr      LSize   Origin Data%
lv_app      vg_lab  owi-aos--- 3.00g
lv_app_snap vg_lab  swi-a-s--- 500.00m lv_app 0.01
Merging of volume vg_lab/lv_app_snap started.
v1
```

5 Look beyond LVM

Optional. Read the RHEL 10 documentation on Stratis. Write two lines on what it adds on top of LVM and XFS, and when you would choose it.

BREAK IT · THE SNAPSHOT THAT OVERFLOWED

Make a snapshot that is too small, then write more data to the origin than the snapshot can hold. Read the snapshot state in `lvs` and explain why the snapshot is now useless.

```
sudo lvcreate -s -n tiny -L 64M /dev/vg_lab/lv_app
sudo dd if=/dev/urandom of=/app/churn.bin bs=1M count=200 oflag=direct
sudo lvs -a vg_lab
sudo lvremove -y vg_lab/tiny; sudo rm /app/churn.bin
```

REFLECTION

Why does XFS's inability to shrink change how you size logical volumes?

When is an LVM snapshot not a backup?

Cleanup. Keep `vg_lab` and `/app` for Labs 23 and 28. Remove `/app/fill.bin`.

Networking Fundamentals on Red Hat

Configure a static network address, set up name resolution, and diagnose a staged connectivity failure using command-line tools.

SYSTEMS

control, node1, node2

CHAPTER

Ch. 14: Networking Fundamentals on Red Hat

OBJECTIVES: BY THE END OF THIS LAB

- Each host has its static address after a reboot
- All three hosts resolve each other by short name
- You wrote down the first failing layer and the evidence for it

1 Read the current configuration

Read the network setup from the kernel's view and from NetworkManager's view.

```
ip -br link; ip -br addr; ip route
nmcli device status
nmcli connection show
sudo cat /etc/NetworkManager/system-connections/*.nmconnection
```

EXPECTED OUTPUT

```
lo          UNKNOWN 127.0.0.1/8 ::1/128
enp1s0     UP        192.168.122.143/24
default via 192.168.122.1 dev enp1s0 proto dhcp src 192.168.122.143 metric 100
```

2 Set a static address

Give each host the static address from the environment table. The example below is for node1.

```
CON=$(nmcli -g NAME connection show --active | head -1)
sudo nmcli con mod "$CON" ipv4.method manual \
  ipv4.addresses 192.168.122.11/24 ipv4.gateway 192.168.122.1 \
  ipv4.dns 192.168.122.1
sudo nmcli con up "$CON"
ip -br addr; ip route
```

EXPECTED OUTPUT

```
Connection successfully activated (D-Bus active path:
/org/freedesktop/NetworkManager/ActiveConnection/3)
enp1s0 UP 192.168.122.11/24
default via 192.168.122.1 dev enp1s0 proto static metric 100
```

3 Name resolution

Add all three hosts to `/etc/hosts` on every machine. Then compare the results from the hosts file and from DNS.

```
printf "%s\n" \
    "192.168.122.10 control.lab.example.test control" \
    "192.168.122.11 node1.lab.example.test node1" \
    "192.168.122.12 node2.lab.example.test node2" | sudo tee -a /etc/hosts
getent hosts node2
cat /etc/resolv.conf
sudo dnf -y install bind-utils && dig +short redhat.com
```

EXPECTED OUTPUT

```
192.168.122.12 node2.lab.example.test node2
nameserver 192.168.122.1
(an IP address)
```

4 Diagnostic tools

Test each layer, and see what is listening.

```
ping -c3 192.168.122.1
traceroute -n redhat.com
ss -tulpn
curl -sI https://access.redhat.com | head -1
```

EXPECTED OUTPUT

```
64 bytes from 192.168.122.1: icmp_seq=1 ttl=64 time=0.31 ms
...
Netid State Local Address:Port
tcp LISTEN 0.0.0.0:22 users:(("sshd",pid=1021,fd=7))
HTTP/2 200
```

BREAK IT · WRONG GATEWAY, BROKEN DNS

A partner changes node1 to use a wrong gateway and DNS server. Work up the layers in order: link, address, local subnet, gateway, remote IP, name. Record the first layer that fails, then fix it.

```
sudo nmcli con mod "$CON" ipv4.gateway 192.168.122.254 ipv4.dns 192.168.122.253
sudo nmcli con up "$CON"
# diagnose, in order:
ip -br link; ip -br addr
ping -c2 node2; ping -c2 192.168.122.1
ping -c2 1.1.1.1; ip route
getent hosts redhat.com
```

REFLECTION

Why could you still ping the real gateway while the configured gateway was wrong?

What order will you check network layers in next time, and why that order?

Cleanup. Keep the static addresses and /etc/hosts entries. Later labs assume them.

Remote Access and Secure Administration

Set up key-based SSH access, harden the SSH server configuration, and transfer files securely between systems.

SYSTEMS

control, node1, node2

CHAPTER

Ch. 15: Remote Access and Secure Administration

OBJECTIVES: BY THE END OF THIS LAB

- ssh node1 and ssh node2 from control need no password
- sshd -T shows passwordauthentication no and permitrootlogin no
- You reattached to the tmux session after disconnecting

1 Keys, not passwords

On control, as student, generate a key and install it on both nodes.

```
ssh-keygen -t ed25519 -C "student@control"  
ssh-copy-id student@node1  
ssh-copy-id student@node2  
ssh node1 hostname
```

EXPECTED OUTPUT

```
Number of key(s) added: 1  
Now try logging into the machine, with: "ssh 'student@node1'"  
node1.lab.example.test
```

2 Client configuration

Put connection defaults in the client config file so you don't type them each time.

```
cat >> ~/.ssh/config <<'EOF'  
Host node1 node2  
  User student  
  IdentityFile ~/.ssh/id_ed25519  
EOF  
chmod 600 ~/.ssh/config
```

3 Harden the server

On node1, keep a second SSH session open as a safety line. Add a drop-in file, then validate it before you reload. Files in `sshd_config.d` are read in name order and the first value set wins, so `40-` takes precedence over `50-redhat.conf`.

```
sudo tee /etc/ssh/sshd_config.d/40-lab-hardening.conf <<'EOF'
PasswordAuthentication no
PermitRootLogin no
MaxAuthTries 3
AllowGroups wheel devs
EOF
sudo sshd -t && sudo systemctl reload sshd
sudo sshd -T | grep -Ei "passwordauth|permitroot|allowgroups"
```

EXPECTED OUTPUT

```
passwordauthentication no
permitrootlogin no
allowgroups wheel
allowgroups devs
```

4 Confirm password logins are refused

From node2, which has no key yet, try a password login. You should get Permission denied (publickey).

```
ssh -o PubkeyAuthentication=no student@node1
```

EXPECTED OUTPUT

```
student@node1: Permission denied (publickey,gssapi-keyex,gssapi-with-mic).
```

5 Move files

Copy files one-off with `scp`, and sync a whole tree with `rsync`.

```
scp /etc/hosts node1:/tmp/
mkdir -p ~/lab15 && date > ~/lab15/stamp
rsync -av ~/lab15/ node2:~/lab15-copy/
```

EXPECTED OUTPUT

```
sending incremental file list
created directory /home/student/lab15-copy
./
stamp
sent 142 bytes  received 69 bytes
```

6 Survive a disconnect

Start a long job inside tmux, cut the connection, then reconnect and reattach.

```
ssh node1
sudo dnf -y install tmux
tmux new -s work      # run: top   then detach with Ctrl-b d
# close the terminal, reconnect, then:
tmux attach -t work
```

BREAK IT · THE KEY THAT STOPPED WORKING

Loosen the permissions on `authorized_keys` on node1. Key login now fails. Find the reason in the server log.

```
chmod 666 ~/.ssh/authorized_keys
# from control: ssh -v node1
sudo journalctl -u sshd -n 20 | grep -i "bad ownership or modes"
chmod 600 ~/.ssh/authorized_keys; chmod 700 ~/.ssh
```

REFLECTION

Why keep a second session open while changing sshd configuration?

What do you lose and gain by disabling password authentication entirely?

Cleanup. Keep the keys and the hardening drop-in. `rm -rf ~/lab15 ~/lab15-copy` on node2.

Configuring Common Services

Install and configure a web server and a file-sharing service, then diagnose a service that is configured correctly but can't be reached.

SYSTEMS

node1, node2

CHAPTER

Ch. 16: Configuring Common Services

OBJECTIVES: BY THE END OF THIS LAB

- From node2, curl http://node1 returns the page
- From node2, /mnt/hello.txt is readable over NFS
- apachectl configtest reports Syntax OK

1 Web server

Install httpd, publish a page and test it locally.

```
sudo dnf -y install httpd
echo "node1 web: lab 16" | sudo tee /var/www/html/index.html
sudo systemctl enable --now httpd
curl http://localhost
```

EXPECTED OUTPUT

```
node1 web: lab 16
```

2 The configuration pattern

Put your change in a drop-in file under conf.d, validate the syntax, reload the service, and check the effect. Every service in RHEL follows this cycle.

```
echo "ServerTokens Prod" | sudo tee /etc/httpd/conf.d/security.conf
sudo apachectl configtest
sudo systemctl reload httpd
curl -sI http://localhost | grep Server
```

EXPECTED OUTPUT

```
Syntax OK
Server: Apache
```

3 File sharing with NFS

Export a directory to the lab network.

```
sudo dnf -y install nfs-utils
sudo mkdir -p /srv/nfs/share && echo shared | sudo tee /srv/nfs/share/hello.txt
echo "/srv/nfs/share 192.168.122.0/24(rw,sync,root_squash)" | sudo tee
/etc/exports.d/lab.exports
sudo systemctl enable --now nfs-server
sudo exportfs -rav
```

EXPECTED OUTPUT

```
exporting 192.168.122.0/24:/srv/nfs/share
```

4 Database

Install MariaDB, secure it, and create a database.

```
sudo dnf -y install mariadb-server
sudo systemctl enable --now mariadb
sudo mysql_secure_installation
sudo mysql -e "CREATE DATABASE labdb; SHOW DATABASES;"
```

EXPECTED OUTPUT

```
+-----+
| Database          |
+-----+
| information_schema |
| labdb              |
| mysql              |
| performance_schema |
+-----+
```

5 Awareness: naming and address services

Read the RHEL 10 documentation on providing DNS (bind or unbound) and DHCP (Kea). For each one, write down its package, config file and systemd unit. It follows the same pattern as task 2.

BREAK IT · WORKS LOCALLY, FAILS REMOTELY

From node2, both the web page and the NFS mount fail, but on node1 both work locally. Check each layer in turn: running, listening, local and remote. Name the blocking layer before you touch anything.

```
# on node2:
curl -m3 http://node1
sudo mount -t nfs node1:/srv/nfs/share /mnt
# on node1:
systemctl is-active httpd nfs-server
ss -tlnp | grep -E ":80 |:2049 "
sudo firewall-cmd --list-services
# fix (explored fully in Lab 17):
sudo firewall-cmd --permanent --add-service={http,nfs} && sudo firewall-cmd --reload
```

REFLECTION

How did "running, listening, local, remote" narrow down the failure?

Which parts of the configuration pattern would work unchanged for a service you have never set up?

Cleanup. Keep httpd, NFS and MariaDB running for Labs 17, 18 and 23.

The Firewall and Network Security

Configure the firewall to allow the services you set up. Prove the firewall was what blocked a service and fix it, then apply default-deny thinking to the whole configuration.

SYSTEMS

node1, node2, control

CHAPTER

Ch. 17: The Firewall and Network Security

OBJECTIVES: BY THE END OF THIS LAB

- From node2, NFS works. From control, it is refused
- Every open public service has a written reason
- Runtime and permanent rules are identical after a reboot

1 Read the firewall

Check the firewall state, the default zone, the active zones and the full rule set.

```
sudo firewall-cmd --state
sudo firewall-cmd --get-default-zone
sudo firewall-cmd --get-active-zones
sudo firewall-cmd --list-all
```

EXPECTED OUTPUT

```
running
public
public
  interfaces: enp1s0
public (active)
  target: default
  services: cockpit dhcpv6-client http nfs ssh
```

2 Runtime versus permanent

Add a rule to the runtime only and reload. It disappears. Then add it permanently.

```
sudo firewall-cmd --add-service=https
sudo firewall-cmd --reload; sudo firewall-cmd --list-services    # gone
sudo firewall-cmd --permanent --add-service=https && sudo firewall-cmd --reload
```

EXPECTED OUTPUT

```
success
success
cockpit dhcpv6-client http nfs ssh
```

3 Scope a service with zones

Limit NFS to node2 only by putting its address in the internal zone.

```
sudo firewall-cmd --permanent --zone=internal --add-source=192.168.122.12/32
sudo firewall-cmd --permanent --zone=internal --add-service=nfs
sudo firewall-cmd --permanent --zone=public --remove-service=nfs
sudo firewall-cmd --reload; sudo firewall-cmd --get-active-zones
```

EXPECTED OUTPUT

```
internal
  sources: 192.168.122.12/32
public
  interfaces: enp1s0
```

4 Firewall or service?

Learn to tell the two errors apart. "No route to host" means the firewall rejected the connection. "Connection refused" means the port is open but nothing is listening on it.

```
# from node2, before opening 8888:
curl -m3 http://node1:8888      # No route to host
sudo firewall-cmd --permanent --add-port=8888/tcp && sudo firewall-cmd --reload
curl -m3 http://node1:8888      # Connection refused (nothing listening yet; see Lab 18)
```

EXPECTED OUTPUT

```
curl: (7) Failed to connect to node1 port 8888: No route to host
success
success
curl: (7) Failed to connect to node1 port 8888: Connection refused
```

5 Apply default-deny thinking

For every service open in the public zone, write down who needs it and why. Remove anything you can't justify.

```
sudo firewall-cmd --list-services
sudo firewall-cmd --permanent --remove-service={cockpit,dhcpv6-client}
sudo firewall-cmd --reload
```

EXPECTED OUTPUT

```
http https ssh
```

BREAK IT • THE RULE THAT DIDN'T PERSIST

Add a runtime-only rule, test that it works, then reboot. Find the difference between runtime and permanent, and promote the rule properly.

```
sudo firewall-cmd --add-service=mysql
sudo systemctl reboot
diff <(sudo firewall-cmd --list-all) <(sudo firewall-cmd --list-all --permanent)
sudo firewall-cmd --runtime-to-permanent
```

REFLECTION

How do "No route to host" and "Connection refused" tell you different things about where the problem is?

Which open service was hardest to justify, and what did you decide?

Cleanup. Keep the zone and port 8888 rules for Lab 18.

SELinux and Mandatory Access Control

Hit a service that SELinux blocks, confirm from the logs that it is an SELinux denial, and fix it properly by correcting the context. Leave SELinux enabled throughout.

SYSTEMS

node1

CHAPTER

Ch. 18: SELinux and Mandatory Access Control

OBJECTIVES: BY THE END OF THIS LAB

- `getenforce` returns Enforcing, and `/etc/selinux/config` says enforcing
- `page.html`, `/srv/` and port 8888 all serve content
- `semanage fcontext -l -C` and `semanage port -l -C` show your local changes

1 Read labels

Check the mode, then look at the labels on files, processes and ports.

```
sudo dnf -y install polycoreutils-python-utils setroubleshoot-server
getenforce; sestatus
ls -Z /var/www/html
ps -eZ | grep httpd
sudo semanage port -l | grep -w http_port_t
```

EXPECTED OUTPUT

```
Enforcing
SELinux status:                enabled
Current mode:                  enforcing
system_u:object_r:httpd_sys_content_t:s0 index.html
system_u:system_r:httpd_t:s0    1502 ? 00:00:00 httpd
http_port_t    tcp  80, 81, 443, 488, 8008, 8009, 8443, 9000
```

2 Denial 1: a file with the wrong label

`mv` keeps a file's original label. Move a page from your home directory into the web root and request it.

```
echo "from home" > ~/page.html
sudo mv ~/page.html /var/www/html/
curl -s localhost/page.html | head -3      # 403
ls -Z /var/www/html/page.html             # user_home_t
sudo ausearch -m AVC -ts recent
sudo journalctl -t setroubleshoot --since "5 min ago"
sudo restorecon -v /var/www/html/page.html
```

EXPECTED OUTPUT

```
(403 Forbidden page)
unconfined_u:object_r:user_home_t:s0 /var/www/html/page.html
type=AVC msg=audit(...): avc: denied { read } for pid=1504 comm="httpd"
name="page.html" scontext=system_u:system_r:httpd_t:s0
tcontext=unconfined_u:object_r:user_home_t:s0 tclass=file
Relabeled /var/www/html/page.html from unconfined_u:object_r:user_home_t:s0 to
unconfined_u:object_r:httpd_sys_content_t:s0
```

3 A custom document root

Serve /srv/web. Record the correct label as a policy rule, then apply it with restorecon.

```
sudo mkdir -p /srv/web && echo "srv web" | sudo tee /srv/web/index.html
printf "Alias /srv /srv/web\n<Directory /srv/web>\n Require all
granted\n</Directory>\n" | sudo tee /etc/httpd/conf.d/srv.conf
sudo semanage fcontext -a -t httpd_sys_content_t "/srv/web(/.*)?"
sudo restorecon -Rv /srv/web
sudo systemctl reload httpd && curl localhost/srv/
```

EXPECTED OUTPUT

```
Relabeled /srv/web from unconfined_u:object_r:var_t:s0 to
unconfined_u:object_r:httpd_sys_content_t:s0
srv web
```

4 Denial 2: a non-standard port

Make httpd listen on 8888. It won't start until the port has the right label. The firewall port was already opened in Lab 17.

```
echo "Listen 8888" | sudo tee /etc/httpd/conf.d/port.conf
sudo systemctl restart httpd      # fails
sudo ausearch -m AVC -ts recent | grep name_bind
sudo semanage port -a -t http_port_t -p tcp 8888
sudo systemctl restart httpd && curl localhost:8888
```

EXPECTED OUTPUT

```
Job for httpd.service failed because the control process exited with error code.
avc: denied { name_bind } for pid=1622 comm="httpd" src=8888
scontext=system_u:system_r:httpd_t:s0 tcontext=system_u:object_r:unreserved_port_t:s0
node1 web: lab 16
```

5 Booleans

List the booleans that control httpd. Explain what httpd_can_network_connect allows and when you would turn it on. Use -P so the setting persists.

```
getsebool -a | grep ^httpd_
sudo semanage boolean -l | grep httpd_can_network_connect
```

EXPECTED OUTPUT

```
httpd_can_network_connect --> off
httpd_enable_homedirs --> off
...
```

BREAK IT • THE DISABLE REFLEX

Count how many times in this lab disabling SELinux would have "fixed" a problem, and how many times you actually needed to. Check that the permanent setting is still enforcing.

```
grep ^SELINUX= /etc/selinux/config
sudo ausearch -m AVC -ts today | grep -c denied
```

REFLECTION

How did you know the 403 was SELinux and not file permissions?

Why is semanage fcontext plus restorecon better than chcon?

Cleanup. Keep the fcontext and port rules. rm /etc/httpd/conf.d/port.conf if you don't want httpd on 8888.

Package Management and Software

Install and update software with the package manager, add a repository, work through a dependency problem, and diagnose a failed installation.

SYSTEMS

node1

CHAPTER

Ch. 19: Package Management and Software

OBJECTIVES: BY THE END OF THIS LAB

- dnf repolist shows both DVD repositories
- dnf history shows your install and the undo
- You can say why rpm -qf returns "not owned" for handmade

1 Query before you install

Search for packages, get details, find which package provides a file, and check which package owns a file already installed.

```
dnf repolist
dnf search tree; dnf info tree
dnf provides "*/bin/dig"
rpm -qf /etc/ssh/sshd_config
```

EXPECTED OUTPUT

```
bind-utils-32:9.18...el10.x86_64 : Utilities for querying DNS name servers
Repo          : rhel-10-for-x86_64-appstream-rpms
Matched from:
Filename      : /usr/bin/dig
openssh-server-9.9p1-...el10.x86_64
```

2 Install, verify, undo

Install two packages, check an installed package's files against the RPM database, and undo the last transaction.

```
sudo dnf -y install tree bind-utils
rpm -ql tree
rpm -V openssh-server          # no output = unmodified
dnf history
sudo dnf history undo last
```

EXPECTED OUTPUT

```
/usr/bin/tree
...
ID | Command line          | Date and time    | Action(s) | Altered
 7 | install -y tree bind-... | 2026-09-23 13:02 | Install   |      4
```

3 Add a local repository

Attach the RHEL DVD to node1 and set it up as a repository.

```
sudo mkdir -p /mnt/rhel10 && sudo mount -o ro /dev/sr0 /mnt/rhel10
sudo tee /etc/yum.repos.d/local-dvd.repo <<'EOF'
[dvd-baseos]
name=RHEL 10 DVD BaseOS
baseurl=file:///mnt/rhel10/BaseOS
gpgcheck=1
gpgkey=file:///etc/pki/rpm-gpg/RPM-GPG-KEY-redhat-release
[dvd-appstream]
name=RHEL 10 DVD AppStream
baseurl=file:///mnt/rhel10/AppStream
gpgcheck=1
gpgkey=file:///etc/pki/rpm-gpg/RPM-GPG-KEY-redhat-release
EOF
dnf --disablerepo="*" --enablerepo="dvd-*" list available | head
```

EXPECTED OUTPUT

```
repo id      repo name
dvd-appstream RHEL 10 DVD AppStream
dvd-baseos    RHEL 10 DVD BaseOS
...
```

4 Read the dependency web

See what httpd pulls in, and what would break if you removed a core library. Answer N at the prompt.

```
dnf repoquery --requires --resolve httpd
dnf repoquery --installed --whatrequires openssl-lib | wc -l
sudo dnf remove openssl-lib      # read the list, then answer N
```

EXPECTED OUTPUT

```
(a long list including apr, apr-util, httpd-core, httpd-filesystem, ...)
214
Error: Problem: The operation would result in removing the following protected packages:
dnf, systemd ...
```

5 Track software installed by hand

Install a file outside the package manager and see what the package database knows about it.

```
echo 'echo hi' | sudo tee /usr/local/bin/handmade && sudo chmod +x  
/usr/local/bin/handmade  
rpm -qf /usr/local/bin/handmade
```

EXPECTED OUTPUT

```
file /usr/local/bin/handmade is not owned by any package
```

BREAK IT • THE UNREACHABLE REPOSITORY

Put a typo in the DVD baseurl. Read the error, confirm the cause with verbose output, and fix it.

```
sudo sed -i "s#/mnt/rhel10/BaseOS#/mnt/rhel10/BaseOS#" /etc/yum.repos.d/local-  
dvd.repo  
sudo dnf makecache  
dnf repolist -v --repo dvd-baseos  
sudo sed -i "s#BaseOS#BaseOS#" /etc/yum.repos.d/local-dvd.repo
```

REFLECTION

What happens to software installed outside dnf when you rebuild the server?

Why did the dnf remove preview matter more than the command itself?

Cleanup. `sudo rm /usr/local/bin/handmade`; leave the DVD repo in place for Lab 31.

Subscription, Content, and System Management at Scale

Register your system, apply security errata, and plan how you would keep a fleet of systems consistent and up to date.

SYSTEMS

control, node1, node2

CHAPTER

Ch. 20: Subscription, Content, and System Management at Scale

OBJECTIVES: BY THE END OF THIS LAB

- node1 shows no outstanding critical or important security errata
- dnf needs-restarting -r is clean (reboot if not)
- Your fleet plan names a detection method for drift and for unregistered hosts

1 Subscription state

Confirm the registration status, the system's identity and its content repositories.

```
sudo subscription-manager status
sudo subscription-manager identity
sudo subscription-manager repos --list-enabled
```

EXPECTED OUTPUT

```
Overall Status: Registered
system identity: 3f1c7a0e-...
+-----+
  Available Repositories in /etc/yum.repos.d/redhat.repo
+-----+
Repo ID:    rhel-10-for-x86_64-baseos-rpms
```

2 Security errata

List the security advisories that apply to this system. Read one, apply it, and check whether a reboot is needed.

```
sudo dnf updateinfo summary
sudo dnf updateinfo list --security
sudo dnf updateinfo info <RHSA-ID>
sudo dnf upgrade --advisory=<RHSA-ID>
sudo dnf needs-restarting -r
```

EXPECTED OUTPUT

```
Updates Information Summary: available
  3 Security notice(s)
    1 Important Security notice(s)
    2 Moderate Security notice(s)
RHSA-2026:xxxx Important/Sec. openssl-libs-...
No core libraries or services have been updated since boot-up.
Reboot should not be necessary.
```

3 Central view (optional)

Register with Red Hat Insights and find node1 in the inventory at console.redhat.com.

```
sudo dnf -y install insights-client
sudo insights-client --register
```

4 A manual inventory

Collect release, kernel and registration state from every host with one loop. Notice how quickly this stops being practical.

```
for h in node1 node2; do
  echo "== $h"; ssh -t $h 'cat /etc/redhat-release; uname -r; sudo subscription-
manager status | grep Overall'
done
```

EXPECTED OUTPUT

```
== node1
Red Hat Enterprise Linux release 10.0 (Coughlan)
6.12.0-55.9.1.el10_0.x86_64
Overall Status: Registered
== node2
...
```

5 Plan for a fleet of 300

Write one page covering: dev, test and production environments; how content is promoted between them; how often you patch; how you detect an unregistered or out-of-date host; and which tools you would use (Satellite, Insights, Ansible). Lab 22 builds the automation part.

BREAK IT · THE HOST THAT FELL OUT OF PATCHING

Unregister node2 and look at what it can no longer see. Then describe how your fleet plan would have caught it.

```
sudo subscription-manager unregister    # on node2
sudo dnf updateinfo list --security
sudo subscription-manager register
```

REFLECTION

How would you know that one host in 300 had stopped receiving updates?

What makes an erratum safe to apply to production?

Cleanup. Make sure node2 is registered again.

Scheduling, Automation, and Scripting

Write a shell script that automates a real administrative task with proper error handling, and schedule it to run reliably.

SYSTEMS

node1

CHAPTER

Ch. 21: Scheduling, Automation, and Scripting

OBJECTIVES: BY THE END OF THIS LAB

- disk-guard.timer is active and its last run succeeded
- lab-clean.sh refuses /etc and defaults to a dry run
- Both scripts use set -euo pipefail and exit non-zero on failure

1 Write a monitoring script

The script warns when any filesystem passes a usage threshold. It logs to the journal and exits non-zero, so systemd sees the failure.

```
sudo tee /usr/local/sbin/disk-guard.sh <<'EOF'
#!/usr/bin/env bash
# disk-guard: warn when a filesystem crosses a usage threshold
set -euo pipefail
threshold="${1:-80}"
status=0
while read -r pct mount; do
    if (( pct >= threshold )); then
        logger -t disk-guard -p user.warning "$mount at $pct% (limit $threshold%)"
        echo "WARN $mount $pct%"
        status=1
    fi
done <<(df --output=pcent,target -x tmpfs -x devtmpfs | tail -n +2 | tr -d "%")
exit "$status"
EOF
sudo chmod 755 /usr/local/sbin/disk-guard.sh
```

2 Test the failure paths

Check the syntax, then run the script with a threshold low enough to trigger it and one too high to trigger. Read the exit codes and the journal entries.

```

bash -n /usr/local/sbin/disk-guard.sh
sudo /usr/local/sbin/disk-guard.sh 10; echo "exit=$?"
sudo /usr/local/sbin/disk-guard.sh 99; echo "exit=$?"
journalctl -t disk-guard -n 5

```

EXPECTED OUTPUT

```

WARN / 12%
WARN /boot 31%
exit=1
exit=0
Sep 23 13:40:02 node1 disk-guard[4011]: / at 12% (limit 10%)

```

3 Write a destructive script that fails safe

This cleanup script refuses to run outside its sandbox and does a dry run unless you tell it otherwise.

```

sudo tee /usr/local/sbin/lab-clean.sh <<'EOF'
#!/usr/bin/env bash
set -euo pipefail
target="${1:?usage: lab-clean.sh DIR}"
days="${DAYS:-7}"
case "$target" in
    /var/tmp/lab/*) ;;
    *) echo "refusing to clean $target" >&2; exit 2 ;;
esac
if [[ "${DRY_RUN:-1}" == 1 ]]; then
    find "$target" -type f -mtime +"$days" -print
else
    find "$target" -type f -mtime +"$days" -print -delete
fi
EOF
sudo chmod 755 /usr/local/sbin/lab-clean.sh
sudo /usr/local/sbin/lab-clean.sh /etc          # refused
sudo mkdir -p /var/tmp/lab/x && sudo touch -d "10 days ago" /var/tmp/lab/x/old
sudo /usr/local/sbin/lab-clean.sh /var/tmp/lab/x # dry run

```

EXPECTED OUTPUT

```

refusing to clean /etc
/var/tmp/lab/x/old

```

4 Schedule it

Run disk-guard every hour with a systemd timer. The cron equivalent is shown for comparison.

```
sudo tee /etc/systemd/system/disk-guard.service <<'EOF'
[Service]
Type=oneshot
ExecStart=/usr/local/sbin/disk-guard.sh 85
EOF
sudo tee /etc/systemd/system/disk-guard.timer <<'EOF'
[Timer]
OnCalendar=hourly
Persistent=true
[Install]
WantedBy=timers.target
EOF
sudo systemctl daemon-reload && sudo systemctl enable --now disk-guard.timer
# cron equivalent: echo "0 * * * * root /usr/local/sbin/disk-guard.sh 85" >
/etc/cron.d/disk-guard
```

EXPECTED OUTPUT

```
Created symlink /etc/systemd/system/timers.target.wants/disk-guard.timer →
/etc/systemd/system/disk-guard.timer.
```

BREAK IT • THE JOB THAT FAILED SILENTLY

Point the service at a script path that doesn't exist, then trigger it. Find two ways you would notice without watching the job run.

```
sudo sed -i "s#disk-guard.sh#disk-gaurd.sh#" /etc/systemd/system/disk-guard.service
sudo systemctl daemon-reload && sudo systemctl start disk-guard.service
systemctl --failed
systemctl list-timers disk-guard.timer
```

REFLECTION

What does `set -euo pipefail` protect against? Name a case where it could surprise you.

Why should a scheduled job fail loudly instead of quietly?

Automation at Scale with Ansible

Write an Ansible playbook that configures a service on several systems. Run it twice to prove it is idempotent, then use it to fix a system that has drifted.

SYSTEMS

control → node1, node2

CHAPTER

Ch. 22: Automation at Scale with Ansible

OBJECTIVES: BY THE END OF THIS LAB

- A second run reports changed=0 on both hosts
- The drifted node2 is back to the declared state
- You can explain why shell tasks break idempotence and what replaces them

1 Set up the control node

Install `ansible-core` on control. The playbook also needs the `ansible.posix` collection for its `firewalld` module.

```
sudo dnf -y install ansible-core
ansible-galaxy collection install ansible.posix
ansible --version
```

EXPECTED OUTPUT

```
ansible [core 2.16.x]
python version = 3.12.x
```

2 Inventory and configuration

Define the managed hosts and the default settings.

```
mkdir -p ~/ansible && cd ~/ansible
cat > inventory.ini <<'EOF'
[web]
node1
node2
EOF
cat > ansible.cfg <<'EOF'
[defaults]
inventory = inventory.ini
remote_user = student
[privilege_escalation]
become = true
EOF
ansible web -m ping
```

EXPECTED OUTPUT

```
node1 | SUCCESS => {
    "changed": false,
    "ping": "pong"
}
node2 | SUCCESS => { ... "ping": "pong" }
```

3 Write the playbook

Describe the state you want, not the steps to get there.

```

cat > web.yml <<'EOF'
- name: Configure the web tier
  hosts: web
  tasks:
    - name: httpd is installed
      ansible.builtin.dnf:
        name: httpd
        state: present
    - name: Server tokens are minimal
      ansible.builtin.copy:
        dest: /etc/httpd/conf.d/security.conf
        content: "ServerTokens Prod\n"
      notify: reload httpd
    - name: Index page is published
      ansible.builtin.copy:
        dest: /var/www/html/index.html
        content: "Managed by Ansible on {{ inventory_hostname }}\n"
        mode: "0644"
    - name: httpd is running and enabled
      ansible.builtin.service:
        name: httpd
        state: started
        enabled: true
    - name: HTTP is allowed through the firewall
      ansible.posix.firewalld:
        service: http
        permanent: true
        immediate: true
        state: enabled
  handlers:
    - name: reload httpd
      ansible.builtin.service:
        name: httpd
        state: reloaded
EOF

```

4 Run it twice

The first run makes changes. The second must report changed=0.

```

ansible-playbook web.yml -K
ansible-playbook web.yml -K          # expect changed=0
curl node1; curl node2

```

EXPECTED OUTPUT

```
PLAY RECAP *****
node1 : ok=6  changed=4  unreachable=0  failed=0
node2 : ok=6  changed=4  unreachable=0  failed=0
(second run)
node1 : ok=5  changed=0  unreachable=0  failed=0
node2 : ok=5  changed=0  unreachable=0  failed=0
Managed by Ansible on node1
Managed by Ansible on node2
```

5 Detect and correct drift

Change node2 by hand, see the drift in check mode, then converge it back.

```
ssh node2 'sudo systemctl stop httpd; echo defaced | sudo tee
/var/www/html/index.html'
ansible-playbook web.yml -K --check --diff
ansible-playbook web.yml -K
```

EXPECTED OUTPUT

```
TASK [Index page is published] ***
--- before: /var/www/html/index.html
+++ after: /var/www/html/index.html
-defaced
+Managed by Ansible on node2
changed: [node2]
TASK [httpd is running and enabled] ***
changed: [node2]
```

BREAK IT · THE TASK THAT ISN'T IDEMPOTENT

Replace the ServerTokens task with a shell append. Run the playbook three times, then count the duplicate lines. Rewrite the task with `ansible.builtin.lineinfile`.

```
- name: Server tokens (bad)
  ansible.builtin.shell: echo "ServerTokens Prod" >> /etc/httpd/conf.d/security.conf

ssh node1 grep -c ServerTokens /etc/httpd/conf.d/security.conf
```

REFLECTION

What exactly makes a task idempotent? Which modules guarantee it and which don't?

How would you run `--check --diff` on a schedule to detect drift?

Cleanup. Keep ~/ansible on control. Labs 27, 29 and 30 extend it.

Backup, Recovery, and System Resilience

Back up your system's configuration and data, recover from a simulated failure, and prove the restore actually worked.

SYSTEMS

node1, node2

CHAPTER

Ch. 23: Backup, Recovery, and System Resilience

OBJECTIVES: BY THE END OF THIS LAB

- sha256sum -c reports OK for every restored file
- httpd serves the original pages after the restore
- A copy of the backup exists on node2 and was test-restored there

1 Decide what matters

List what a rebuild of node1 would need: /etc, /var/www, /srv, /app, a database dump, the package list, and the storage layout (LVM metadata is backed up automatically under /etc/lvm/backup). For each item, note how you would get it back.

2 Back up configuration and data

Back up /etc with its SELinux labels, ACLs and extended attributes. Also record checksums and the package list.

```
sudo mkdir -p /backup
sudo tar --selinux --acls --xattrs -czpf /backup/etc-$(date +%F).tar.gz /etc
sudo tar --selinux --acls --xattrs -czpf /backup/www-$(date +%F).tar.gz /var/www /srv
sudo find /var/www/html -type f -exec sha256sum {} + | sudo tee /backup/www.sha256
rpm -qa --qf "%{NAME}\n" | sort | sudo tee /backup/packages.txt > /dev/null
sudo mysqldump --all-databases | sudo tee /backup/db.sql > /dev/null
```

3 Snapshot-consistent backup

Back up /app from a frozen LVM snapshot instead of the live filesystem.

```
sudo lvcreate -s -n app_bk -L 500M /dev/vg_lab/lv_app
sudo mount -o ro,nouuid /dev/vg_lab/app_bk /mnt/snap
sudo tar -czpf /backup/app-$(date +%F).tar.gz -C /mnt/snap .
sudo umount /mnt/snap && sudo lvremove -y vg_lab/app_bk
```

EXPECTED OUTPUT

```
Logical volume "app_bk" created.  
Logical volume "app_bk" successfully removed.
```

4 Copy it off the host

A backup that lives only on the machine it protects isn't a backup.

```
sudo tar -cf /tmp/node1-backup.tar -C / backup && sudo chown student /tmp/node1-backup.tar  
scp /tmp/node1-backup.tar node2:~/
```

EXPECTED OUTPUT

```
node1-backup.tar          100%   38MB  95.1MB/s   00:00
```

5 Simulate the failure and restore

Destroy the web configuration and content. Restore from the archive, fix the labels, and check the result against the checksums.

```
sudo rm -rf /etc/httpd/conf/httpd.conf /var/www/html/*  
sudo systemctl restart httpd      # fails  
sudo tar -xzf /backup/etc-*.tar.gz -C / etc/httpd  
sudo tar -xzf /backup/www-*.tar.gz -C / var/www  
sudo restorecon -Rv /etc/httpd /var/www  
sudo systemctl restart httpd && sudo sha256sum -c /backup/www.sha256
```

EXPECTED OUTPUT

```
Job for httpd.service failed because the control process exited with error code.  
Relabeled /var/www/html/index.html from ... to ...httpd_sys_content_t:s0  
/var/www/html/index.html: OK  
/var/www/html/page.html: OK
```

BREAK IT • THE BACKUP NOBODY TESTED

On node2, restore the copied archive into a scratch directory and compare it with node1. Record anything missing or different. Until you have done this, you don't know that your backup works.

```
mkdir -p /tmp/restore-test && tar -xf ~/node1-backup.tar -C /tmp/restore-test  
tar -tzf /tmp/restore-test/backup/etc-*.tar.gz | wc -l  
ssh node1 "sudo find /etc | wc -l"
```

REFLECTION

What is your recovery time and recovery point for node1 now, measured rather than guessed?

What did the test restore find that the backup itself didn't show?

Cleanup. Keep /backup. `rm -rf /tmp/restore-test` on node2.

Security Hardening and Compliance

Apply a security baseline, harden accounts, services and the network, and audit the system against a compliance standard.

SYSTEMS

node1

CHAPTER

Ch. 24: Security Hardening and Compliance

OBJECTIVES: BY THE END OF THIS LAB

- The after-scan score is higher than the before-scan score, with the remaining failures explained
- SELinux is enforcing and the firewall allows only justified services
- auditd records changes to /etc/sudoers.d

1 Scan the baseline

Take a snapshot first. Scan node1 against the CIS Server Level 1 profile and record the score.

```
sudo dnf -y install openscap-scanner scap-security-guide
DS=/usr/share/xml/scap/ssg/content/ssg-rhel10-ds.xml
oscap info $DS | grep -i "profile" | head -20
sudo oscap xccdf eval --profile xccdf_org.ssgproject.content_profile_cis_server_l1 \
  --results /root/scan-before.xml --report /root/scan-before.html $DS
```

EXPECTED OUTPUT

```
Profiles:
  Title: CIS Red Hat Enterprise Linux 10 Benchmark for Level 1 - Server
  Id: xccdf_org.ssgproject.content_profile_cis_server_l1
...
```

2 Harden accounts

Enforce password quality and lockout after failed logins, lock accounts that aren't used, and set password ageing.

```
sudo authselect current
sudo authselect enable-feature with-faillock
echo "minlen = 14" | sudo tee /etc/security/pwquality.conf.d/50-lab.conf
sudo usermod -L carol
sudo chage -M 90 -W 14 alice && sudo chage -l alice
```

EXPECTED OUTPUT

```
profile-id: local
Enabled features:
- with-faillock
Maximum number of days between password change      : 90
Number of days of warning before password expires   : 14
```

3 Reduce the attack surface

Review what is enabled and listening. Disable anything this server doesn't need, and harden kernel network settings.

```
systemctl list-unit-files --type=service --state=enabled
ss -tulpn
printf "net.ipv4.conf.all.send_redirects = 0\nnet.ipv4.conf.all.accept_redirects = 0\n" | sudo tee /etc/sysctl.d/90-lab.conf
sudo sysctl --system | grep redirects
```

EXPECTED OUTPUT

```
net.ipv4.conf.all.send_redirects = 0
net.ipv4.conf.all.accept_redirects = 0
```

4 Audit changes to privilege

Watch the sudoers files with auditd and make the rule persistent.

```
echo "-w /etc/sudoers.d/ -p wa -k sudoers" | sudo tee /etc/audit/rules.d/lab.rules
sudo augenrules --load
sudo touch /etc/sudoers.d/probe && sudo rm /etc/sudoers.d/probe
sudo ausearch -k sudoers -i | tail
```

EXPECTED OUTPUT

```
type=SYSCALL ... comm=touch exe=/usr/bin/touch key=sudoers
type=PATH ... name=/etc/sudoers.d/probe nametype=CREATE
```

5 Remediate selectively and rescan

Generate the remediation as a playbook and read it before you apply anything. Blanket remediation can lock you out. Apply only what you understand, then rescan.

```
sudo oscap xccdf generate fix --fix-type ansible \
  --profile xccdf_org.ssgproject.content_profile_cis_server_l1 \
  --output /root/cis-fix.yml $DS
sudo oscap xccdf eval --profile xccdf_org.ssgproject.content_profile_cis_server_l1 \
  --results /root/scan-after.xml --report /root/scan-after.html $DS
```

BREAK IT · THE DRIFT BACK TO INSECURE

Weeks later, someone adds a drop-in file that re-enables root login. Find it with `sshd -T` and a rescan. Then write down how the Lab 22 approach would have prevented it.

```
echo "PermitRootLogin yes" | sudo tee /etc/ssh/sshd_config.d/01-drift.conf
sudo systemctl reload sshd && sudo sshd -T | grep permitroot
sudo rm /etc/ssh/sshd_config.d/01-drift.conf && sudo systemctl reload sshd
```

REFLECTION

Which remediation did you decide not to apply, and why?

How will this system stay hardened a year from now?

Cleanup. Keep the hardening. If you are locked out of anything: `sudo virsh snapshot-revert node1 lab-24`.

Containers on Red Hat

Run and build containers with Red Hat's own tools, run a container as a service managed by systemd, and diagnose a container that won't start.

SYSTEMS

node1

CHAPTER

Ch. 25: Containers on Red Hat

OBJECTIVES: BY THE END OF THIS LAB

- After a reboot, curl localhost:8081 returns "Built on RHEL" with nobody logged in
- httpd inside the container runs as your UID on the host
- You can explain what :Z does and why it was needed

1 Install the tools

Podman runs without a daemon, and here it runs as an ordinary user rather than root.

```
sudo dnf -y install container-tools
podman version
podman info --format "{{.Host.Security.Rootless}}"
```

EXPECTED OUTPUT

```
Client:      Podman Engine
Version:     5.x.x
true
```

2 Run and inspect

Run a Red Hat UBI httpd image as student and look inside it. The image listens on port 8080.

```
podman run -d --name web -p 8080:8080 registry.access.redhat.com/ubi9/httpd-24
podman ps; curl -s localhost:8080 | head -3
podman logs web | tail -5
podman exec -it web cat /etc/redhat-release
podman rm -f web
```

EXPECTED OUTPUT

CONTAINER ID	IMAGE	STATUS	PORTS
1a2b3c4d5e6f	registry.access.redhat.com/ubi9/httpd-24:latest	Up 3 seconds	
0.0.0.0:8080->8080/tcp	web		
(HTML of the default test page)			
Red Hat Enterprise Linux release 9.x (Plow)			

3 Build an image

Build an image from a Containerfile and run it.

```
mkdir -p ~/labweb && cd ~/labweb
echo "Built on RHEL" > index.html
cat > Containerfile <<'EOF'
FROM registry.access.redhat.com/ubi9/httpd-24
COPY index.html /var/www/html/index.html
EOF
podman build -t localhost/labweb:1 .
podman images
```

EXPECTED OUTPUT

```
STEP 1/2: FROM registry.access.redhat.com/ubi9/httpd-24
STEP 2/2: COPY index.html /var/www/html/index.html
COMMIT localhost/labweb:1
REPOSITORY          TAG      IMAGE ID      SIZE
localhost/labweb    1        9f8e7d6c5b4a  380 MB
```

4 See rootless at work

Map users inside the container to the host UIDs they become.

```
podman run -d --name labweb -p 8081:8080 localhost/labweb:1
podman top labweb user huser
podman unshare cat /proc/self/uid_map
podman rm -f labweb
```

EXPECTED OUTPUT

USER	HUSER
default	100000
0	1000
1	100000
	65536

5 Run it as a systemd service (Quadlet)

Declare the container as a unit so it starts at boot, without any login session.

```

mkdir -p ~/.config/containers/systemd
cat > ~/.config/containers/systemd/labweb.container <<'EOF'
[Unit]
Description=Lab web container
[Container]
Image=localhost/labweb:1
PublishPort=8081:8080
[Install]
WantedBy=default.target
EOF
systemctl --user daemon-reload
systemctl --user start labweb; systemctl --user status labweb
sudo loginctl enable-linger student
sudo systemctl reboot      # then: curl localhost:8081

```

EXPECTED OUTPUT

```

• labweb.service - Lab web container
   Loaded: loaded (/home/student/.config/containers/systemd/labweb.container;
   generated)
   Active: active (running)
   Built on RHEL

```

BREAK IT • THE CONTAINER THAT WON'T SERVE

Two classic rootless failures. First, a port below 1024. Second, a bind mount that doesn't carry an SELinux label. Diagnose each one from the error message, the logs and the AVC records.

```

podman run -d --name low -p 80:8080 localhost/labweb:1      # permission denied
mkdir -p ~/site && echo hi > ~/site/index.html
podman run -d --name bad -p 8082:8080 -v ~/site:/var/www/html
registry.access.redhat.com/ubi9/httpd-24
curl -s localhost:8082 | head -3; sudo ausearch -m AVC -ts recent | tail -3
podman rm -f bad && podman run -d --name good -p 8082:8080 -v ~/site:/var/www/html:Z
registry.access.redhat.com/ubi9/httpd-24

```

REFLECTION

How does rootless Podman change what an attacker gets if they break out of a container?

Why does the container need a systemd unit to survive a reboot?

Cleanup. `podman rm -f low good; systemctl --user disable --now labweb` if you don't want it running.

OpenShift and Kubernetes on Red Hat

Deploy a containerized application to OpenShift, and trace how the Linux skills from this guide apply underneath the platform.

SYSTEMS

OpenShift Local or Developer Sandbox

CHAPTER

Ch. 26: OpenShift and Kubernetes on Red Hat

OBJECTIVES: BY THE END OF THIS LAB

- Both routes serve content
- You explained the Pending pod from its events alone
- On crc: you found kubelet, CRI-O and SELinux container_t on the node

1 Choose an environment

Pick one. OpenShift Local (crc) runs on your own machine and needs a large host (check the current requirements; plan for at least 4 cores and 10 GiB free RAM). The Developer Sandbox is free and runs in the browser, but it doesn't give you node access.

```
crc setup && crc start
eval $(crc oc-env)
oc login -u developer https://api.crc.testing:6443
```

EXPECTED OUTPUT

```
Login successful.
You have one project on this server: "demo"
```

2 Deploy from an image

Create a project, deploy the UBI httpd image, and expose it through a route.

```
oc new-project lab26
oc new-app --name hello registry.access.redhat.com/ubi9/httpd-24
oc get pods -w
oc expose service/hello
curl -s http://$(oc get route hello -o jsonpath="{.spec.host}") | head -3
```

EXPECTED OUTPUT

```
deployment.apps "hello" created
service "hello" created
hello-6d9c... 1/1 Running 0 20s
route.route.openshift.io/hello exposed
(HTML of the test page)
```

3 Deploy from source

Build straight from a Git repository using Source-to-Image.

```
oc new-app httpd~https://github.com/sclorg/httpd-ex --name ex
oc logs -f buildconfig/ex
oc expose service/ex
```

EXPECTED OUTPUT

```
Cloning "https://github.com/sclorg/httpd-ex" ...
STEP 1/9: FROM registry.redhat.io/ubi9/httpd-24...
Push successful
```

4 Operate it

Inspect, scale and read events. These are the same reading skills as Lab 11, applied to the platform.

```
oc describe pod -l deployment=hello | tail -20
oc scale deployment/hello --replicas=3 && oc get pods -o wide
oc get events --sort-by=.lastTimestamp | tail
```

EXPECTED OUTPUT

```
deployment.apps/hello scaled
hello-6d9c...-abcde 1/1 Running 0 5s 10.217.0.61 crc
hello-6d9c...-fghij 1/1 Running 0 5s 10.217.0.62 crc
```

5 Look at the Linux underneath (crc only)

Log in as kubeadmin, open a debug shell on the node, and use the tools from this guide on the host.

```
oc debug node/$(oc get nodes -o name | cut -d/ -f2)
chroot /host
systemctl status kubelet crio --no-pager | head -20
journalctl -u kubelet -n 10 --no-pager
getenforce; ps -eZ | grep container_t | head -3
```

EXPECTED OUTPUT

```
• kubelet.service - Kubernetes Kubelet
  Active: active (running)
• crio.service - Container Runtime Interface for OCI (CRI-O)
  Active: active (running)
Enforcing
system_u:system_r:container_t:s0:c12,c47 ...
```

BREAK IT • THE POD THAT WON'T SCHEDULE

Ask for more CPU than the cluster has. Diagnose the Pending pod from its events, then fix the request.

```
oc set resources deployment/hello --requests=cpu=64
oc get pods; oc describe pod -l deployment=hello | grep -A3 Events
oc set resources deployment/hello --requests=cpu=100m
```

REFLECTION

Which Lab 11 reading skills did you use on the Pending pod?

What in OpenShift is plainly Linux underneath: processes, SELinux labels, systemd units?

Cleanup. `oc delete project lab26`

Red Hat in the Cloud and Hybrid Infrastructure

Launch and configure a RHEL instance in the cloud, and manage it with the same administration and automation skills as your lab machines.

SYSTEMS

Cloud account + control

CHAPTER

Ch. 27: Red Hat in the Cloud and Hybrid Infrastructure

OBJECTIVES: BY THE END OF THIS LAB

- The playbook converges cloud1 with changed=0 on its second run
- `curl http://<public-ip>` works from your IP only
- The instance is terminated with no billable resources left behind

1 Launch an instance

Choose AWS, Azure or Google Cloud. Use a RHEL 10 image: either pay-as-you-go, or bring your own subscription through Red Hat Cloud Access. Pick the smallest size, use an ed25519 key pair, and allow SSH only from your own IP. The example is for AWS.

```
aws ec2 run-instances --image-id <rhel10-ami> --instance-type t3.micro \  
  --key-name lab --security-group-ids <sg-id> \  
  --user-data file://user-data.yaml \  
  --tag-specifications "ResourceType=instance,Tags=[{Key=Name,Value=cloud1},  
{Key=purpose,Value=lab27}]"
```

2 Configure at first boot with cloud-init

Pass this file as user data. It runs on the first boot. Afterwards, check its results.

```
#cloud-config  
hostname: cloud1  
package_upgrade: true  
packages: [httpd]  
runcmd:  
  - systemctl enable --now httpd  
  
# after boot:  
ssh ec2-user@<public-ip>  
cloud-init status --long  
sudo tail /var/log/cloud-init-output.log
```

EXPECTED OUTPUT

```
status: done
extended_status: done
Cloud-init v. 24.x finished at ... Datasource DataSourceEc2Local. Up 41.2 seconds
```

3 Apply the same skills

Run the checks from earlier labs on the instance. Note where the cloud replaces a layer, such as security groups versus firewallld.

```
hostnamectl; getenforce; lsblk; df -h
systemctl is-active firewallld httpd
sudo subscription-manager status
```

EXPECTED OUTPUT

```
Static hostname: cloud1
Enforcing
inactive        (firewalld, on many cloud images)
active          (httpd)
```

4 Manage it with Ansible

Add the instance to your inventory and run the Lab 22 playbook against it. Open port 80 in the security group, but only to your IP.

```
cat >> ~/ansible/inventory.ini <<'EOF'
[web]
cloud1 ansible_host=<public-ip> ansible_user=ec2-user
EOF
ansible-playbook web.yml -l cloud1
```

EXPECTED OUTPUT

```
PLAY RECAP ***
cloud1 : ok=5  changed=2  unreachable=0  failed=1  (firewalld task, see Break it)
```

5 Control cost

Set a budget alert. Confirm the tags are there. Terminate the instance when you finish, and check that no volumes or IP addresses are left over.

```
aws ec2 terminate-instances --instance-ids <id>
aws ec2 describe-volumes --filters Name=status,Values=available
```

BREAK IT - AUTOMATION THAT ASSUMED THE LAB

If firewalld isn't running on the cloud image, the playbook's firewall task fails. Decide on a fix: install and enable firewalld in the playbook, or skip the task where the cloud firewall does the job. Implement it and rerun.

REFLECTION

Which layer did the cloud take over from you, and which ones stayed yours?

How would you stop an unused instance from costing money for months?

Cleanup. Terminate the instance and delete its key pair and security group.

Troubleshooting Red Hat Systems

Diagnose staged problems across the layers: a failed service, an SELinux denial, a full filesystem and a firewall-blocked service. Identify the root cause of each.

SYSTEMS

node1, node2 (+ partner)

CHAPTER

Ch. 28: Troubleshooting Red Hat Systems

OBJECTIVES: BY THE END OF THIS LAB

- curl http://node1 from node2 returns the page
- Each of the four root causes is recorded with its evidence
- The fixes corrected causes, not symptoms: SELinux enforcing, firewall rule permanent, config file removed

1 Stage the faults

Take a snapshot first. A partner runs this on node1 without showing you. If you work alone, have someone else save it as a script so you don't read it. It creates four stacked faults.

```
echo "DocumentRoot /var/www/html" > /etc/httpd/conf.d/zz-legacy.conf
chcon -t user_home_t /var/www/html/index.html
fallocate -l $(( $(df --output=avail -B1 /app | tail -1) - 4096 )) /app/.journal.tmp
firewall-cmd --permanent --remove-service=http && firewall-cmd --reload
```

2 Apply the method

The report is "the website is down". For every step, write down the symptom, the scope, your hypothesis, the test, the result, the fix and how you verified it. Don't change anything until you have a hypothesis.

3 The service layer

Start with whether the service is running. Use Chapters 9 and 11.

```
systemctl status httpd
journalctl -u httpd -b --no-pager | tail
sudo apachectl configtest
```

EXPECTED OUTPUT

```
AH00526: Syntax error on line 1 of /etc/httpd/conf.d/zz-legacy.conf:
Invalid command 'DocumentRooot', perhaps misspelled or defined by a module not included
in the server configuration
```

4 The permission and SELinux layer

Once the service runs but returns 403, check whether ordinary permissions or SELinux is the cause. Use Chapters 6 and 18.

```
namei -l /var/www/html/index.html
ls -Z /var/www/html/index.html
sudo ausearch -m AVC -ts recent
```

EXPECTED OUTPUT

```
unconfined_u:object_r:user_home_t:s0 /var/www/html/index.html
type=AVC ... denied { read } ... tcontext=...user_home_t:s0 tclass=file
```

5 The storage layer

Find what is filling the application volume. Use Chapters 12 and 13.

```
df -h /app; sudo du -xah /app | sort -h | tail -3
```

EXPECTED OUTPUT

```
/dev/mapper/vg_lab-lv_app 3.0G 3.0G 4.0K 100% /app
3.0G /app/.journal.tmp
```

6 The network and firewall layer

It works locally but fails from node2. Use Chapters 14 and 17.

```
# from node2:
curl -m3 http://node1
# on node1:
ss -tlnp | grep :80; sudo firewall-cmd --list-services
```

EXPECTED OUTPUT

```
curl: (7) Failed to connect to node1 port 80: No route to host
LISTEN 0 511 *:80 *:~ users:(("httpd",pid=...))
https ssh
```

7 Write your checklist

Turn what you did into a one-page personal checklist covering boot, services, permissions, SELinux, storage, network and security. Start from Appendix A.

REFLECTION

Which fault hid another one, and how did the layered method reveal it?

Where did you want to act before you had a hypothesis?

Cleanup. `sudo virsh snapshot-revert node1 lab-28`, or keep your fixed system

A Complete Red Hat Build End to End

Build a complete, secured, automated server from a bare installation, using the users, storage, services, security, automation and backup work from the whole guide.

SYSTEMS

New VM prod1, control, node2

CHAPTERCh. 29: A Complete Red Hat Build End to End

OBJECTIVES: BY THE END OF THIS LAB

- Every acceptance check passes after a reboot
- site.yml rebuilds prod1 from a bare install with no manual steps except install and register
- You recorded the rebuild time

1 Install and register

Install a new VM, prod1 (192.168.122.20), from the ISO. Register it and patch it fully. Chapters 2, 19 and 20.

2 Accounts and access

Create a group webadmins and a user deploy in it. deploy logs in with a key only, and its sudo rights are limited to managing httpd. Root SSH is off. Chapters 6, 7 and 15.

3 Storage

Create vg_data across two new disks, with lv_web (2 GiB) mounted at /srv/web and lv_backup (1 GiB) at /backup. Both mount persistently. Chapters 12 and 13.

4 Services

httpd serves /srv/web on port 80 with correct SELinux labels. MariaDB listens only on localhost. Chapters 16 and 18.

5 Security

firewalld allows only ssh and http. SELinux is enforcing. An OpenSCAP CIS L1 scan is saved, with each remaining failure justified. Chapters 17, 18 and 24.

6 Automation

Everything after installation is captured in ~/ansible/site.yml on control. A second run reports changed=0. Chapter 22.

7 Backup

A nightly systemd timer backs up /etc, /srv/web and a database dump to /backup, and copies the archive to node2. Do one test restore. Chapters 21 and 23.

8 Acceptance test

Reboot prod1, then run these checks from control.

```
ssh prod1 'systemctl is-active httpd mariadb firewallld; getenforce'
ssh prod1 'findmnt /srv/web /backup; sudo firewall-cmd --list-services'
ssh prod1 'systemctl list-timers | grep -i backup'
curl -s http://prod1/ | head -1
ansible-playbook site.yml -l prod1 --check
```

EXPECTED OUTPUT

```
active
active
active
Enforcing
TARGET      SOURCE                                FSTYPE
/srv/web    /dev/mapper/vg_data-lv_web           xfs
/backup     /dev/mapper/vg_data-lv_backup        xfs
http ssh
... backup.timer backup.service
(first line of your site)
prod1 : ok=NN  changed=0  unreachable=0  failed=0
```

BREAK IT • THE REBUILD TEST

Destroy prod1. Reinstall it, run site.yml, and restore the data from node2. Time it. That time is your real recovery time.

REFLECTION

Which part of the build took longest, and could automation shorten it?

If prod1 were lost tonight, which step of the recovery are you least sure of?

Cleanup. Keep prod1 as your reference build.

When Red Hat Systems Fail

Audit a system against the failure catalog, find which failures it is most at risk of, and decide what to change.

SYSTEMS

node1 or prod1

CHAPTER

Ch. 30: When Red Hat Systems Fail

OBJECTIVES: BY THE END OF THIS LAB

- The risk register has evidence (command output) for every entry
- The top three risks are fixed and re-checked

1 Foundation failures

Check for config files changed outside any record, and for world-writable files.

```
sudo rpm -Va --nomtime | grep "^..5" | grep " c "
sudo find / -xdev -type f -perm -0002 2>/dev/null
```

EXPECTED OUTPUT

```
S.5....T.  c /etc/dnf/dnf.conf
(no world-writable files)
```

2 Service failures

Look for failed units, and for services that are running but not enabled. Those won't come back after a reboot.

```
systemctl --failed
comm -23 \
  <(systemctl list-units --type=service --state=running --no-legend | awk '{print $1}' | sort) \
  <(systemctl list-unit-files --type=service --state=enabled --no-legend | awk '{print $1}' | sort)
```

EXPECTED OUTPUT

```
UNIT LOAD ACTIVE SUB DESCRIPTION
0 loaded units listed.
(running-but-not-enabled units, if any)
```

3 Storage failures

Check fullness, fstab correctness and room to grow.

```
df -h; df -i
sudo findmnt --verify
sudo vgs -o vg_name,vg_size,vg_free; sudo lvs
```

4 Security failures

Check SELinux, sudo rules, SSH settings and the firewall.

```
getenforce; grep ^SELINUX= /etc/selinux/config
sudo grep -rh "ALL" /etc/sudoers.d/
sudo sshd -T | grep -E "permitroot|passwordauth"
sudo firewall-cmd --list-all
```

5 Automation and scale failures

Check for scripts without safety settings, drift from the playbook, registration and outstanding errata.

```
grep -L "set -euo pipefail" /usr/local/sbin/*.sh
ansible-playbook ~/ansible/site.yml --check --diff    # from control
sudo subscription-manager status
sudo dnf updateinfo list --security | wc -l
```

6 Write the risk register

For each finding, record the failure, the evidence, how likely it is, its impact and the fix. Rank the top three and fix them.

REFLECTION

Which failure category was your system most exposed to, and why?

Which of the three big ideas (file, process, text) helped most in the audit?

Cleanup. Nothing to clean up. Keep the risk register.

Preparing for Red Hat Certification

Practice certification-style tasks under time pressure, confirm your configuration persists across a reboot, and assess how ready you are.

SYSTEMS

node2 (reset to base), node1

CHAPTERCh. 31: Preparing for Red Hat Certification

OBJECTIVES: BY THE END OF THIS LAB

- Every passing task survived the reboot
- You timed yourself and recorded a readiness score
- Every failed task is mapped to a lab to repeat

1 Set the conditions

Revert node2 to its base snapshot. Set a 2.5-hour timer. Use man pages and `/usr/share/doc`, and nothing else. Don't reboot until the end.

2 The task set

Complete as many as you can: (1) Static IP 192.168.122.12/24, gateway and DNS, hostname node2.lab.example.test. (2) BaseOS and AppStream repositories from the DVD. (3) Group sysops with members harry and natasha, and user sarah with a non-interactive shell. (4) `/common/sysops` owned by sysops and setgid, with no access for others. (5) sysops members may run any command with sudo. (6) natasha runs logger "EX practice" every 2 minutes. (7) Volume group exam with 8 MiB extents and a 40-extent ext4 LV data, mounted at `/exam/data`. (8) Resize data to 500 MiB. (9) 512 MiB of extra swap, persistent. (10) httpd on port 82 serving `/var/www/html`, with SELinux and the firewall configured. (11) Automount node1:/srv/nfs/share at `/netdir/share` with autofs. (12) Copy every file owned by harry to `/root/harry-files`. (13) Save the lines of `/etc/services` containing "tcp" to `/root/lines`. (14) Create a bzip2 archive of `/usr/local` at `/root/local.tar.bz2`. (15) Run a rootless container as natasha, started by systemd at boot. (16) Apply the tuned profile it recommends. (17) Reset the root password.

3 Reboot and verify

Reboot once. Check everything, because a change that didn't persist scores zero.

```
ip -br addr; hostname; dnf repolist
id harry; id natasha; getent passwd sarah
ls -ld /common/sysops; sudo -l -U harry
findmnt /exam/data; sudo lvs exam; swapon --show
curl -s localhost:82 | head -1; ls /netdir/share
sudo -iu natasha systemctl --user is-active <unit>
tuned-adm active
```

4 Score and plan

Mark each task pass or fail. For each fail, note whether the cause was knowledge, speed or persistence, and name the lab to redo. Repeat the set in a week and aim for all 17 in under 2 hours.

REFLECTION

Were your failures about knowledge, speed or persistence?

Which task would you most want to be able to do without the manual?

Cleanup. Revert node2 to its base snapshot.

Building Skills and Where to Go Next

Assess your skills against the whole guide, choose a specialization, and write a development plan that includes ongoing practice in your lab.

SYSTEMS

None: planning

CHAPTER

Ch. 32: Building Skills and Where to Go Next

OBJECTIVES: BY THE END OF THIS LAB

- Your self-assessment cites evidence for every rating
- Your 90-day plan has a measurable outcome for each 30-day phase
- The weekly lab session is on your calendar

1 Assess yourself

For each Part (I–VI), rate yourself from 1 to 4: 1 = followed the steps, 2 = did it with notes, 3 = did it from memory, 4 = could diagnose it when broken. Use your Lab 31 results as evidence.

2 Choose a direction

Choose one: system administration, automation and DevOps, containers and OpenShift, cloud, or security. Write down the next credential for it (for example RHCE for Ansible automation, or an OpenShift administration specialist exam). Check its current objectives on [redhat.com](https://www.redhat.com).

3 Keep the lab alive

Schedule a weekly one-hour session: revert to a snapshot, break one thing from the Lab 30 catalog, and fix it. Keep a running log of what broke and how you found it.

4 Write the 90-day plan

Days 1–30: close the gaps in your weakest Part and repeat Lab 31. Days 31–60: build a project in your chosen specialization, managed with Ansible. Days 61–90: sit the exam or ship the project, and publish a write-up. Give each phase a measurable outcome.

5 Follow the platform

Subscribe to the RHEL release notes and security advisories. Follow CentOS Stream to see what is coming next in RHEL. Pick one community to take part in.

REFLECTION

Which chapter changed how you think, not just what you can type?

What will you have to show for yourself at day 90?

Cleanup. Nothing to clean up.

Layered troubleshooting checklist

Start at the layer that matches the symptom, then move one layer at a time. At each layer, write down what you expected to see and what you actually saw. Add to this checklist as you work through the labs. Lab 28 asks you to make it your own.

Boot Does it reach a target?	<pre>journalctl --list-boots journalctl -b -1 -p err systemctl get-default; systemctl --failed systemd-analyze critical-chain</pre>
Service Running, enabled, healthy?	<pre>systemctl status <unit> systemctl is-enabled <unit> journalctl -u <unit> -b --no-pager tail -30 <daemon> -t # e.g. apachectl configtest, sshd -t</pre>
Permissions Can this identity reach this path?	<pre>id <user> namei -l /full/path/to/file stat -c "%A %U:%G %n" <path></pre>
SELinux Is policy denying it?	<pre>getenforce sudo ausearch -m AVC -ts recent ls -Z <path>; ps -eZ grep <proc> sudo restorecon -Rv <path> # fix the label, keep enforcing</pre>
Storage Space, inodes, mounts?	<pre>df -h; df -i findmnt --verify sudo du -xh --max-depth=2 <mount> sort -h tail sudo lsof +L1 # deleted but still open sudo vgs; sudo lvs</pre>
Network Which layer fails first?	<pre>ip -br link; ip -br addr; ip route ping -c2 <gateway>; ping -c2 <remote-ip> getent hosts <name> ss -tulpn grep <port></pre>
Firewall "No route to host" or "refused"?	<pre>sudo firewall-cmd --get-active-zones sudo firewall-cmd --list-all --zone=<zone> sudo firewall-cmd --list-all --permanent # compare runtime vs permanent</pre>

Packages

Is the software what you
think it is?

```
rpm -qf <file>; rpm -V <pkg>  
dnf history; dnf repolist -v  
sudo dnf updateinfo list --security
```

Command quick reference

These are the commands the labs use most, grouped by task. Use them as reminders. The man page is the full reference.

Help

`man <cmd>`
Manual page; / to search

`man -k <word>`
Search manual page descriptions

`<cmd> --help`
Short usage summary

`type <cmd>`
Builtin, alias or file?

Files

`ls -la; ls -li`
List with hidden files and inodes

`cp -a; mv -i; rm -I`
Copy (preserving), move, delete safely

`find <dir> -name/-type/-size`
Locate files

`ln; ln -s`
Hard and symbolic links

`namei -l <path>`
Permissions along a path

Text

`less; head; tail -f`
Read files

`grep -Ei; cut; sort; uniq -c`
Search, filter, count

`awk; sed`
Transform text

`> >> 2> 2>&1 | tee`
Redirect and pipe

Users

`useradd; usermod -aG; passwd`
Manage accounts

`chmod; chown; chgrp`
Permissions and ownership

`visudo -f /etc/sudoers.d/<f>`
Edit sudo rules safely

`sudo -l -U <user>`
Show a user's sudo rights

Processes

`ps -eo ... --sort=-%cpu`
Top consumers

`top; kill -TERM; renice`
Watch, signal, reprioritise

`jobs; bg; fg; Ctrl-Z`
Shell job control

systemd

`systemctl status|start|enable --now`
Manage units

`systemctl --failed; list-timers`
Failures and schedules

`journalctl -u <unit> -b -p err`
Filter the journal

`systemd-analyze blame`
Boot timing

Storage

`lsblk; blkid; findmnt --verify`
Devices, UUIDs, fstab check

`parted; mkfs.xfs; mkswap`
Partition and format

`pvcreate; vgcreate; lvcreate`
Build LVM

`lvextend -r -L +<size>`
Grow an LV and its filesystem

`df -h; du -xh; lsof +L1`
Find the space

Network

`ip -br addr; ip route`
Addresses and routes

`nmcli con mod|up`
Configure connections

`getent hosts; dig`
Name resolution

`ss -tulpn; tracepath`
Listeners and paths

Security

`firewall-cmd --list-all [--permanent]`
Firewall state

`firewall-cmd --permanent --add-service=<s>`
Open a service

`ls -Z; ps -eZ; ausearch -m AVC`
SELinux labels and denials

`semanage fcontext -a; restorecon -Rv`
Fix labels persistently

`sshd -t; sshd -T`
Validate and dump SSH config

Software

`dnf install|remove|history undo`
Manage packages

`dnf provides; rpm -qf; rpm -V`
Who owns a file, what changed

`dnf updateinfo list --security`
Security errata

Containers

`podman run -d -p H:C <img>`
Run a container

`podman ps; logs; exec -it`
Inspect

`podman build -t <tag> .`
Build from a Containerfile

`~/.config/containers/systemd/*.container`
Quadlet units

Automation

`ansible <group> -m ping`
Reach managed hosts

`ansible-playbook <pb> --check --diff`
Preview changes

`bash -n <file>`
Syntax-check a shell file